

Clustering Analysis of Seed Varieties

Sandaru Welikala, Laura Granada, Diego Diaz, Julio Real

2024-11-09

The SEEDS dataset contains measurements of the geometrical properties of kernels belonging to three different varieties of wheat: Kama, Rosa, and Canadian. A total of 210 kernels (70 from each variety) were examined using a non-destructive soft X-ray technique, which allowed high-quality visualization of the internal kernel structure. This technique is significantly more affordable than other imaging methods like scanning microscopy or laser technology. The dataset consists of seven real-valued attributes: area, perimeter, compactness, length of kernel, width of kernel, asymmetry coefficient, and length of kernel groove. All measurements were obtained from wheat grain harvested from experimental fields at the Institute of Agrophysics of the Polish Academy of Sciences in Lublin.

Size: 210 rows (instances) and 7 columns (features). Variables:

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
library(cluster)
library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.4.2
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(GGally)
```

```
## Warning: package 'GGally' was built under R version 4.4.2
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg      ggplot2
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
## intersect, setdiff, setequal, union
```

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
```

```
## corrplot 0.95 loaded
```

```
library(stats)
library(NbClust)
library(fpc)
```

```
## Warning: package 'fpc' was built under R version 4.4.2
```

```
library(clValid)
```

```
## Warning: package 'clValid' was built under R version 4.4.2
```

```
column_names <- c("Area", "Perimeter", "Compactness", "Length of kernel", "Width of kernel", "Asymmetry")
Seeds <- read.csv("../data/seeds_dataset.txt", sep = ",", header = FALSE, col.names = column_names)
head(Seeds)
```

```
##      Area Perimeter Compactness Length.of.kernel Width.of.kernel
## 1 15.26      14.84      0.8710          5.763          3.312
## 2 14.88      14.57      0.8811          5.554          3.333
## 3 14.29      14.09      0.9050          5.291          3.337
## 4 13.84      13.94      0.8955          5.324          3.379
## 5 16.14      14.99      0.9034          5.658          3.562
## 6 14.38      14.21      0.8951          5.386          3.312
##      Asymmetry.coefficient Length.of.kernel.groove Variety
## 1              2.221              5.220              1
## 2              1.018              4.956              1
## 3              2.699              4.825              1
## 4              2.259              4.805              1
## 5              1.355              5.175              1
## 6              2.462              4.956              1
```

Explanatory Data Analysis (EDA)

```
summary(Seeds)
```

```
##      Area      Perimeter      Compactness      Length.of.kernel
## Min.   :10.59  Min.   :12.41  Min.   :0.8081  Min.   :4.899
## 1st Qu.:12.27  1st Qu.:13.45  1st Qu.:0.8569  1st Qu.:5.262
## Median :14.36  Median :14.32  Median :0.8734  Median :5.524
```

```
## Mean :14.85 Mean :14.56 Mean :0.8710 Mean :5.629
## 3rd Qu.:17.30 3rd Qu.:15.71 3rd Qu.:0.8878 3rd Qu.:5.980
## Max. :21.18 Max. :17.25 Max. :0.9183 Max. :6.675
## Width.of.kernel Asymmetry.coefficient Length.of.kernel.groove Variety
## Min. :2.630 Min. :0.7651 Min. :4.519 Min. :1
## 1st Qu.:2.944 1st Qu.:2.5615 1st Qu.:5.045 1st Qu.:1
## Median :3.237 Median :3.5990 Median :5.223 Median :2
## Mean :3.259 Mean :3.7002 Mean :5.408 Mean :2
## 3rd Qu.:3.562 3rd Qu.:4.7687 3rd Qu.:5.877 3rd Qu.:3
## Max. :4.033 Max. :8.4560 Max. :6.550 Max. :3
```

```
unique(Seeds$Variety)
```

```
## [1] 1 2 3
```

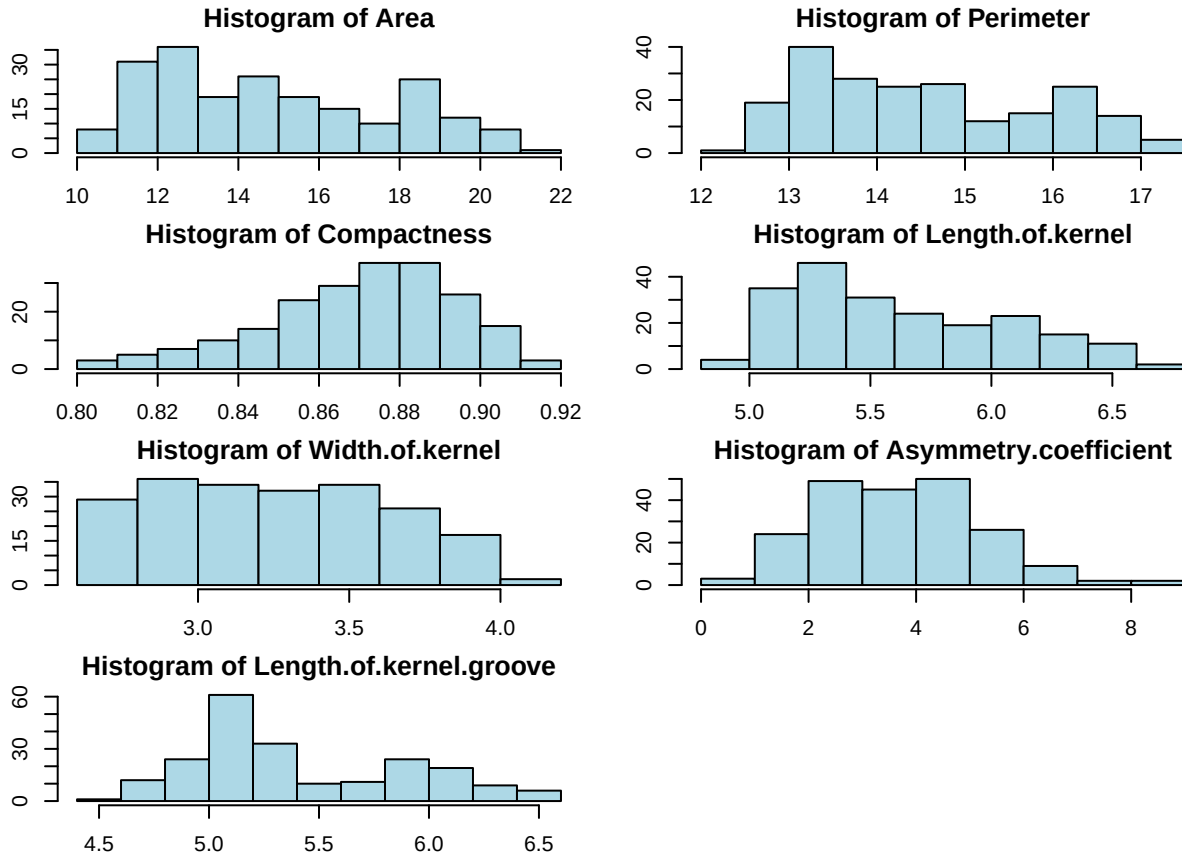
The Variety column in the SEEDS dataset indicates the three distinct wheat varieties: Kama, Rosa, and Canadian, represented numerically as 1, 2, and 3.

```
colSums(is.na(Seeds))
```

```
## Area Perimeter Compactness
## 0 0 0
## Length.of.kernel Width.of.kernel Asymmetry.coefficient
## 0 0 0
## Length.of.kernel.groove Variety
## 0 0
```

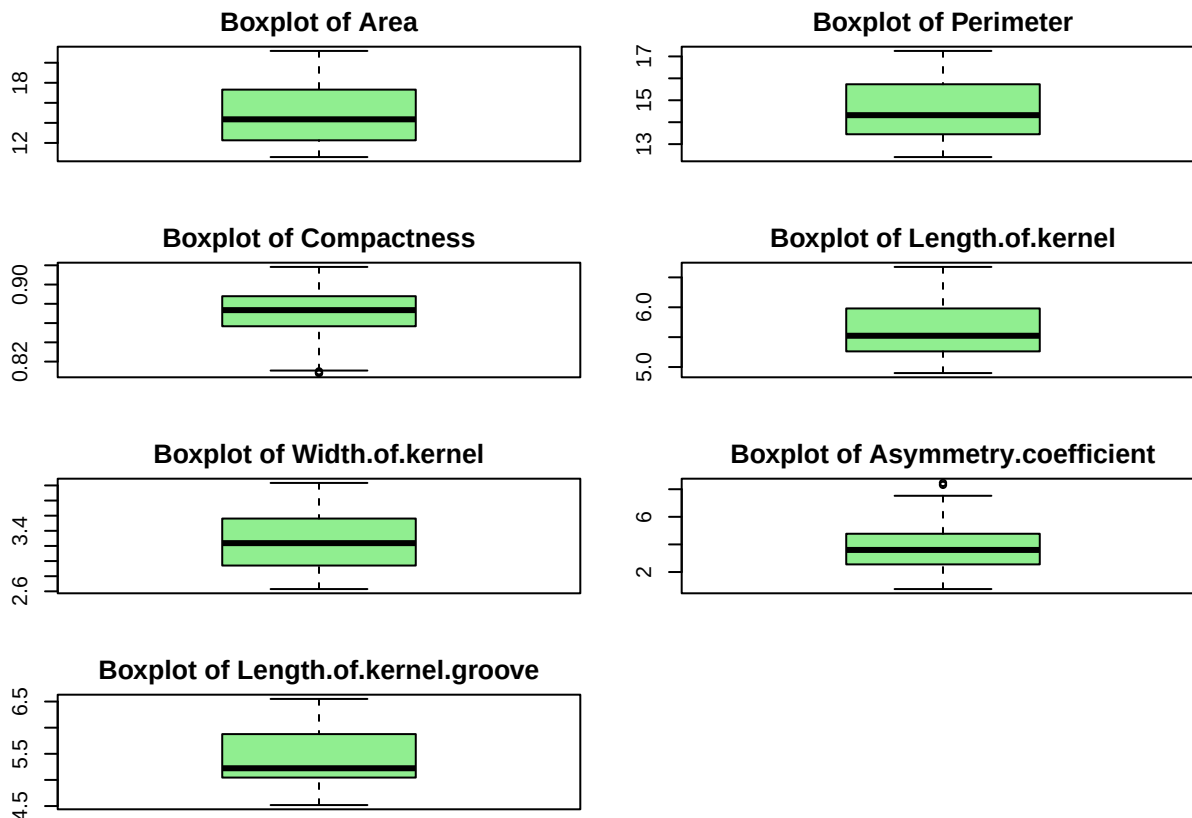
```
old_par <- par(no.readonly = TRUE) #graph parameters
par(mfrow = c(4, 2), mar = c(2, 2, 2, 2)) #4x2 grid

# histograms for each feature to observe distributions
for (i in 1:(ncol(Seeds) - 1)) { # Exclude the 'Variety' column for plots
  hist(Seeds[[i]], main = paste("Histogram of", colnames(Seeds)[i]), xlab = colnames(Seeds)[i], col = "red")
}
```



V1 (Area): The distribution is approximately uniform with no strong skewness. It shows a spread across a range of values. V2 (Perimeter): Slightly skewed to the left, indicating a concentration of kernels with lower perimeter values. V3 (Compactness): Has a fairly normal distribution, showing a balanced spread with a slight peak around the center. V4 (Length of Kernel): Slightly skewed to the right, suggesting more instances with smaller kernel lengths. V5 (Width of Kernel): Appears somewhat uniform with a mild right skew, showing a broad range of kernel widths. V6 (Asymmetry Coefficient): Has a pronounced right skew, indicating many kernels have lower asymmetry coefficients, with fewer high values. V7 (Length of Kernel Groove): Shows a relatively even distribution with a slight right skew. V8 (Class Label): Appears as a categorical variable with equal counts for each class, as expected for a balanced dataset.

```
# Box plots to detect outliers
par(mfrow = c(4, 2), mar = c(2, 2, 2, 2))
for (i in 1:(ncol(Seeds) - 1)) {
  boxplot(Seeds[[i]], main = paste("Boxplot of", colnames(Seeds)[i]), col = "lightgreen")
}
```



Area (V1): No major outliers; K-means and K-means++ should perform stably. Perimeter (V2): Balanced, no significant outliers; minimal impact on clustering stability. Compactness (V3): One minor outlier; could slightly affect K-means, handled better by K-medoids. Length of Kernel (V4): No outliers; all methods should be stable here. Width of Kernel (V5): Uniform, no outliers; stable clustering expected. Asymmetry Coefficient (V6): Contains high-end outliers; K-medoids should manage these better than K-means or K-means++. Length of Kernel Groove (V7): Even, no significant outliers; stable across methods. Variety (V8): Categorical; useful for evaluating clustering accuracy after applying the methods.

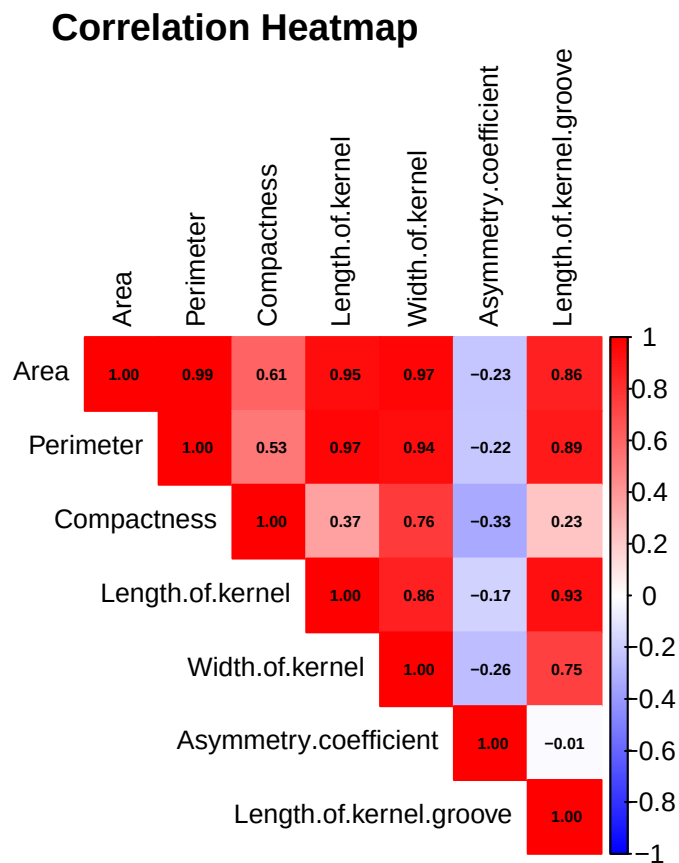
```
cor_matrix <- cor(Seeds[, 1:7], use = "complete.obs")
cor_matrix
```

	Area	Perimeter	Compactness	Length.of.kernel
## Area	1.0000000	0.9943409	0.6082884	0.9499854
## Perimeter	0.9943409	1.0000000	0.5292436	0.9724223
## Compactness	0.6082884	0.5292436	1.0000000	0.3679151
## Length.of.kernel	0.9499854	0.9724223	0.3679151	1.0000000
## Width.of.kernel	0.9707706	0.9448294	0.7616345	0.8604149
## Asymmetry.coefficient	-0.2295723	-0.2173404	-0.3314709	-0.1715624
## Length.of.kernel.groove	0.8636927	0.8907839	0.2268248	0.9328061

	Width.of.kernel	Asymmetry.coefficient
## Area	0.9707706	-0.22957233
## Perimeter	0.9448294	-0.21734037
## Compactness	0.7616345	-0.33147087
## Length.of.kernel	0.8604149	-0.17156243
## Width.of.kernel	1.0000000	-0.25803655
## Asymmetry.coefficient	-0.2580365	1.00000000

```
## Length.of.kernel.groove      0.7491315      -0.01107902
##                               Length.of.kernel.groove
## Area                        0.86369275
## Perimeter                   0.89078390
## Compactness                 0.22682482
## Length.of.kernel            0.93280609
## Width.of.kernel             0.74913147
## Asymmetry.coefficient       -0.01107902
## Length.of.kernel.groove     1.00000000
```

```
corrplot(cor_matrix, method = "color", type = "upper",
         col = colorRampPalette(c("blue", "white", "red"))(200),
         tl.col = "black", tl.cex = 0.8,
         addCoef.col = "black", # Correlation numbers in black
         number.cex = 0.5, # Size of numbers
         title = "Correlation Heatmap",
         mar = c(0, 0, 1, 0))
```



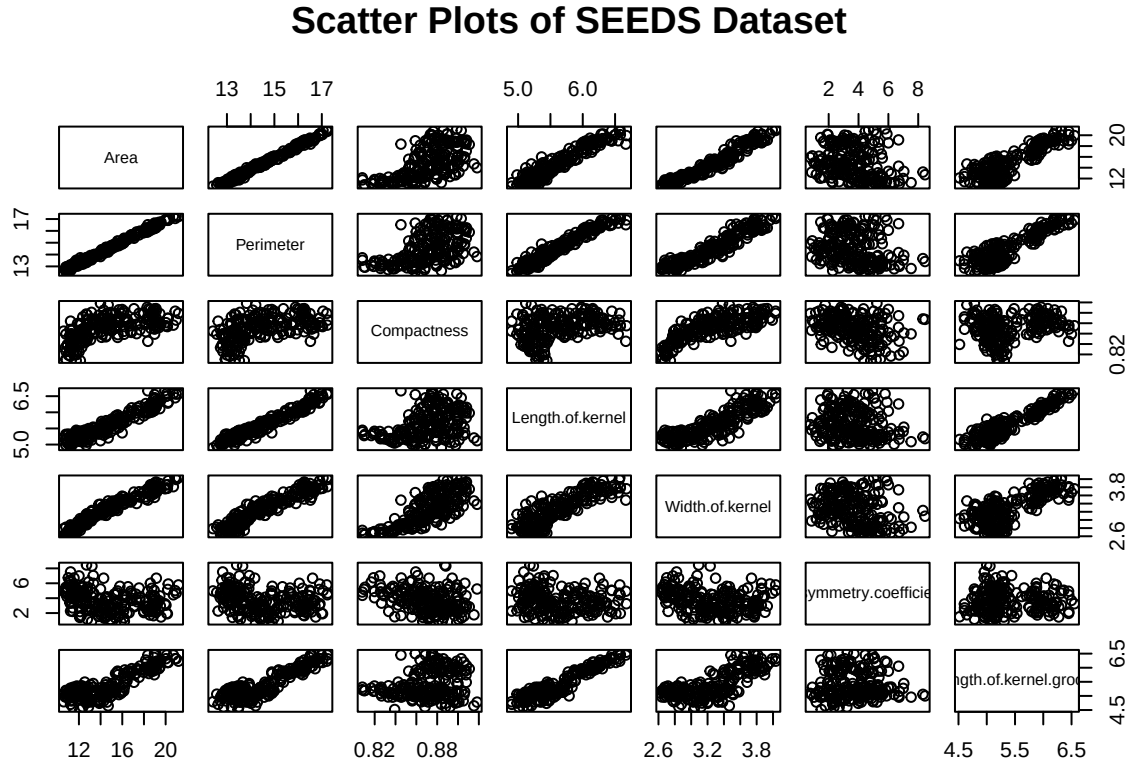
According the correlation matrix we can identify that Area and Perimeter, Area and Length.of.kernal, Area and Width.of.kernal, Perimeter and Length.of.kernal, Perimeter and Width.of.kernal, Length.of.kernal and Length.of.kernal.groove has a high positive correlation.

The Compactness and Width.of.kernal, Area and Length.of.kernal.groove has moderate positive correlation. Asymmetry.coefficient has weak negative correlation with all other variables.

- Variables like Area, Perimeter, Length of kernel, and Width of kernel are highly correlated with one another, likely because they measure related aspects of the kernel size.

- Asymmetry coefficient has weak or negative correlations with most variables, implying it may capture a distinct feature not directly tied to kernel size or shape.
- Compactness has moderate correlations with Width.of.kernel and Area but lower correlations with Length.of.kernel and Length.of.kernel.groove, suggesting that compactness is more closely related to the overall size and width of the kernel rather than its length or groove characteristics.

```
pairs(Seeds[, 1:7], main = "Scatter Plots of SEEDS Dataset")
```



Strong Linear Relationships: The scatter plots between Area and Perimeter, as well as between Length of kernel and Perimeter, show strong positive linear relationships. These relationships suggest that these features may contribute similarly when forming clusters in K-means, K-medoids, and K-means++.

Non-linear Patterns: Some plots, such as Compactness vs. Area, show more scattered points with non-linear distributions. This may influence clustering accuracy as non-linear relationships can affect centroid placement in K-means.

Clusters and Overlaps: While some pairs like Length of kernel and Width of kernel show potential natural group separations, others like Asymmetry coefficient vs. Compactness appear more overlapped, indicating possible challenges in distinguishing clusters with these features.

Outlier Presence: A few plots, such as Asymmetry coefficient vs. other features, show points that might be outliers. Outliers can influence K-means and K-means++ results more significantly than K-medoids, as K-medoids is generally more robust to outliers.

```
Seeds$Variety <- as.factor(Seeds$Variety)
```

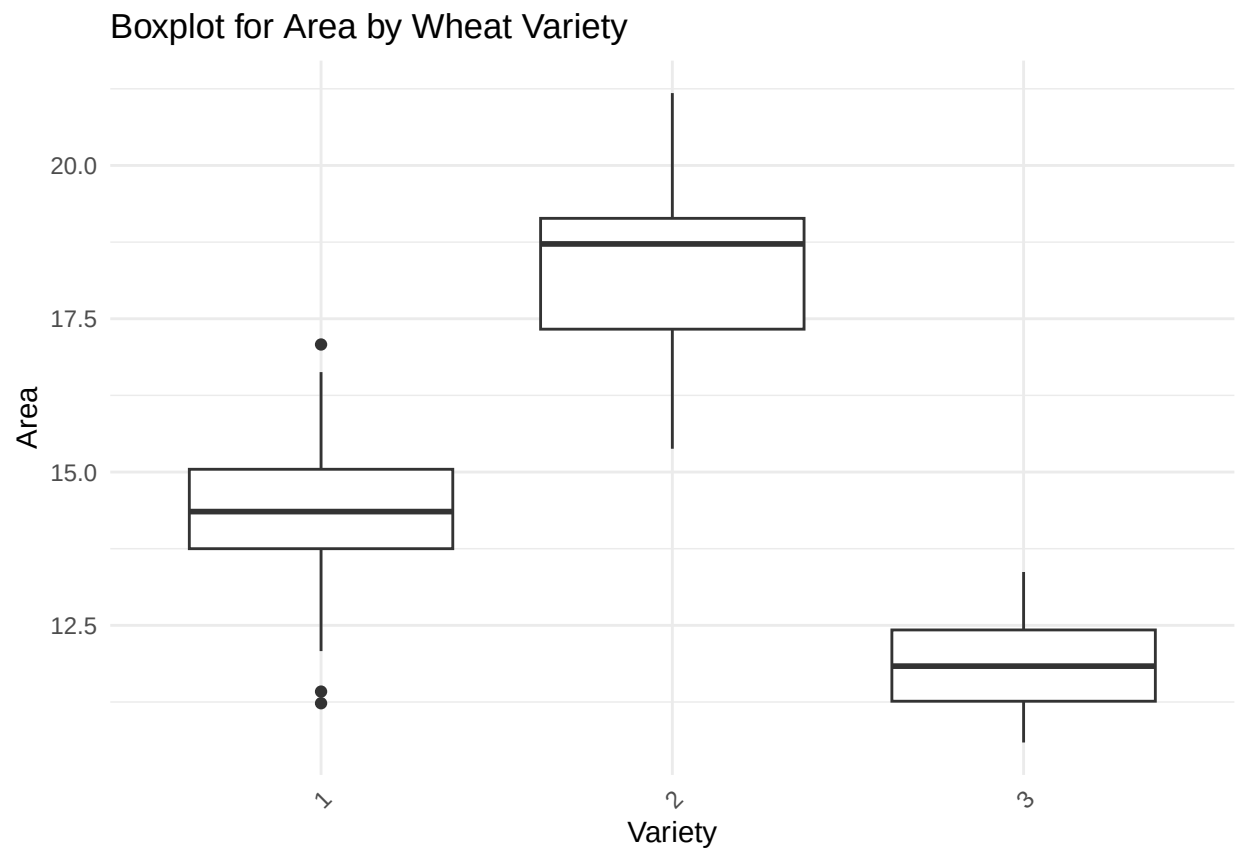
```

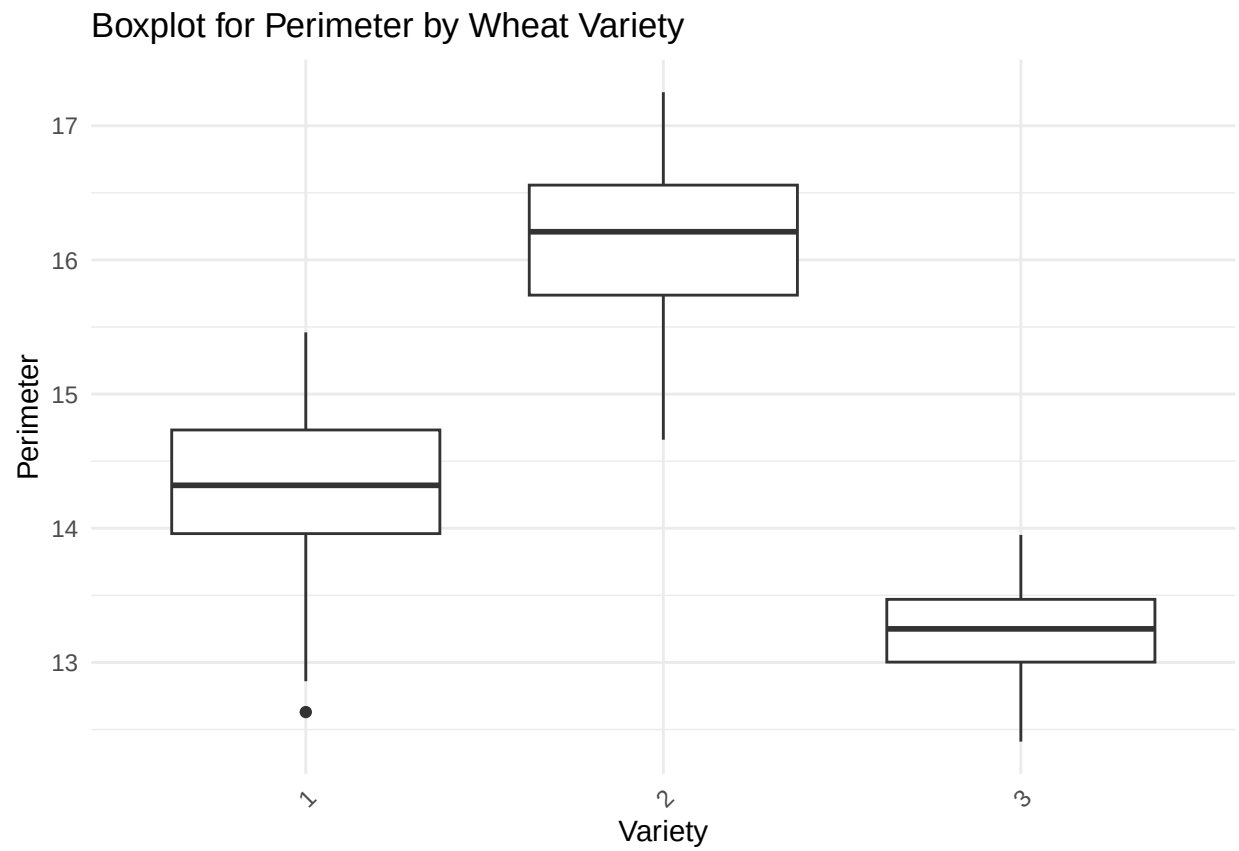
variable_names <- colnames(Seeds)[1:7] # Select the first 7 variables

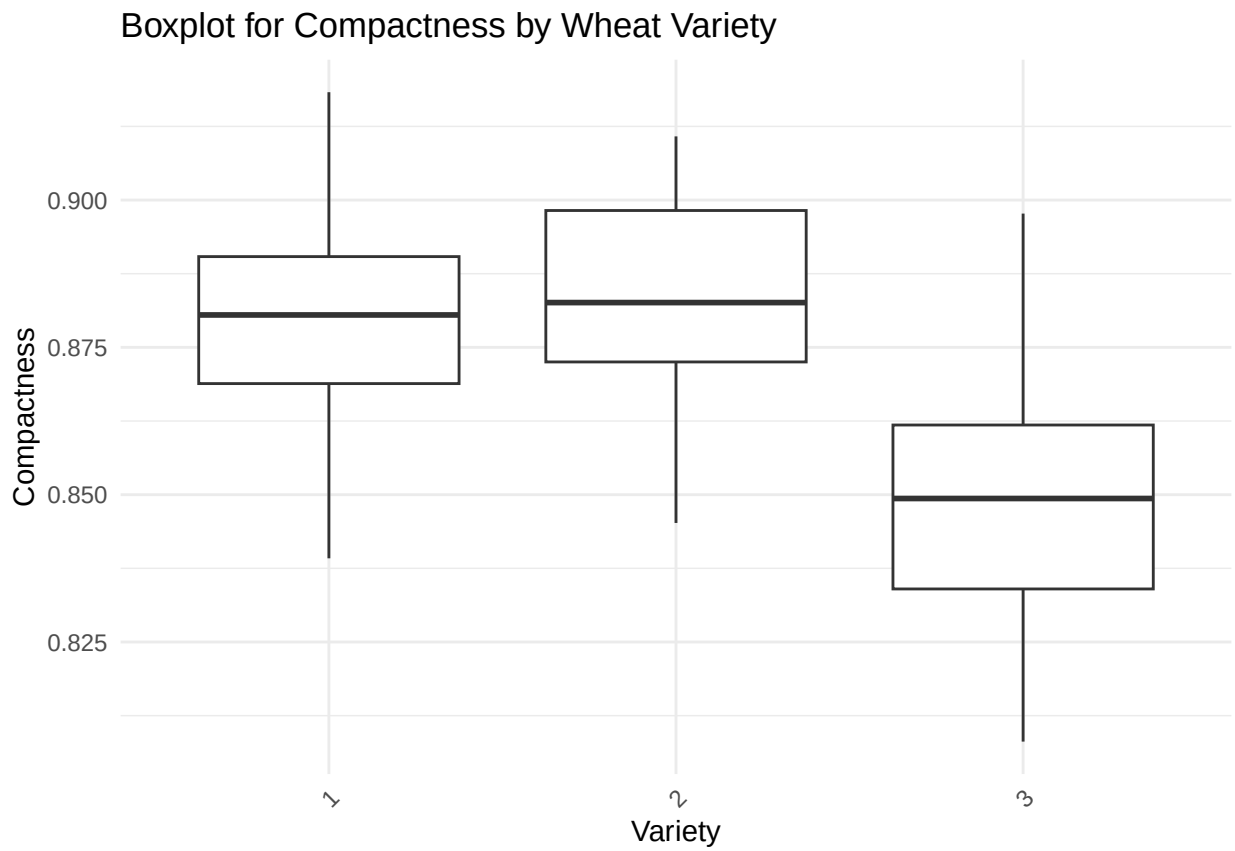
for (var in variable_names) {
  p <- ggplot(Seeds, aes(x = Variety, y = .data[[var]])) +
    geom_boxplot() +
    ggtitle(paste("Boxplot for", var, "by Wheat Variety")) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))

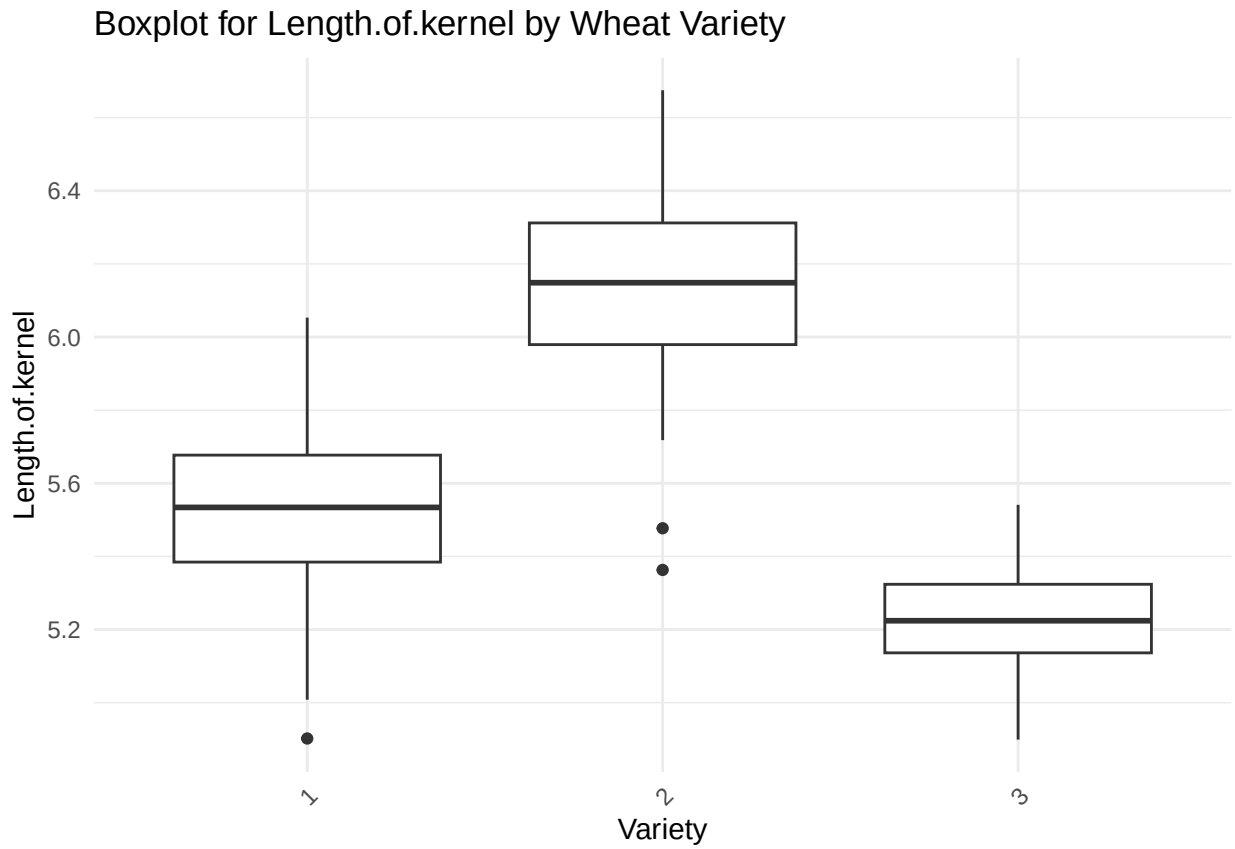
  print(p)
}

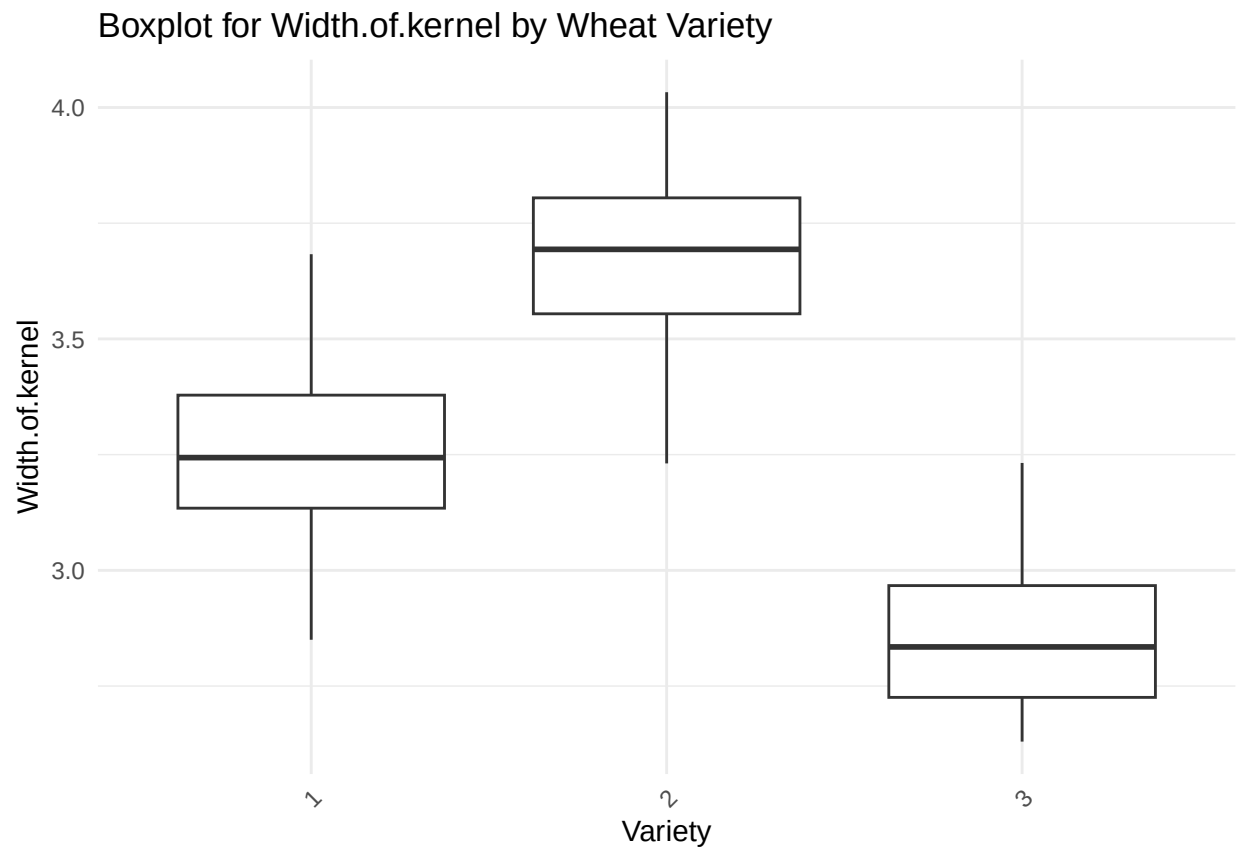
```

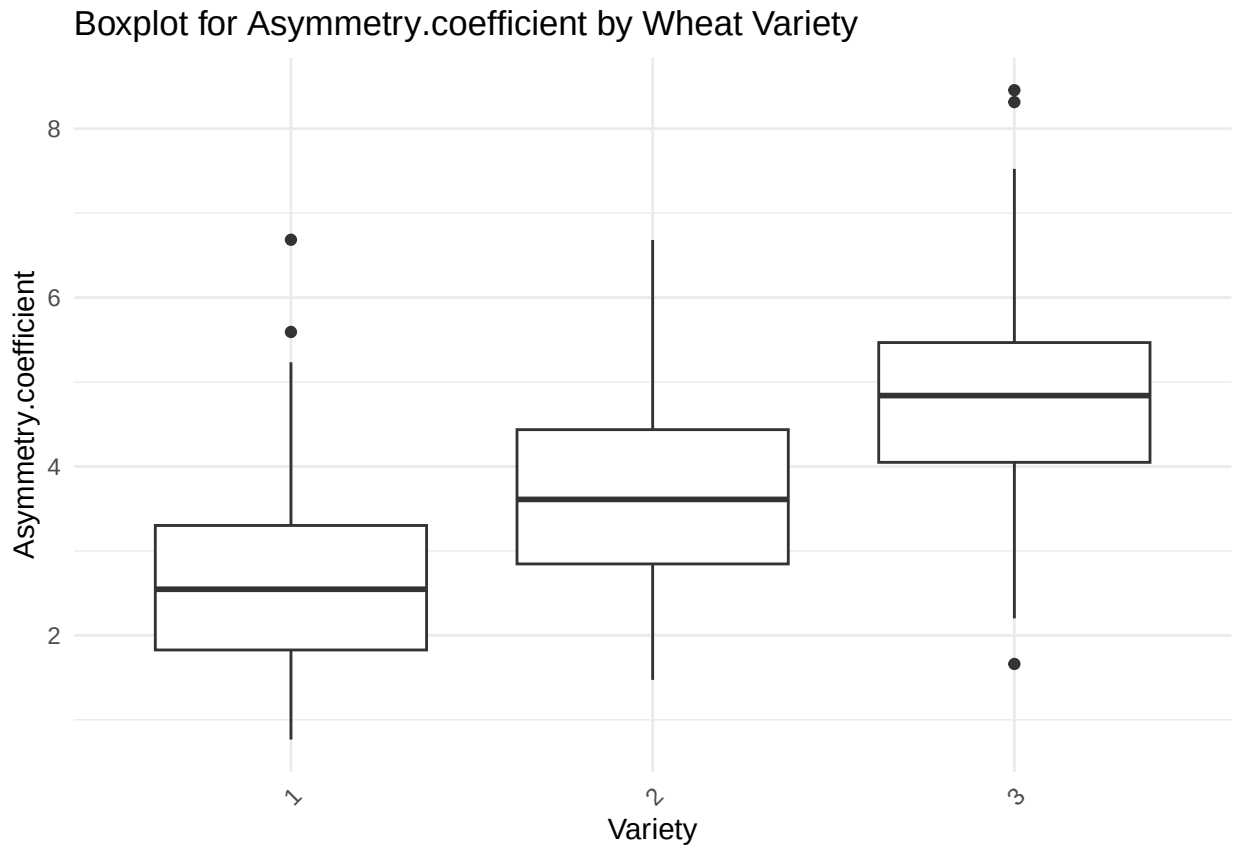


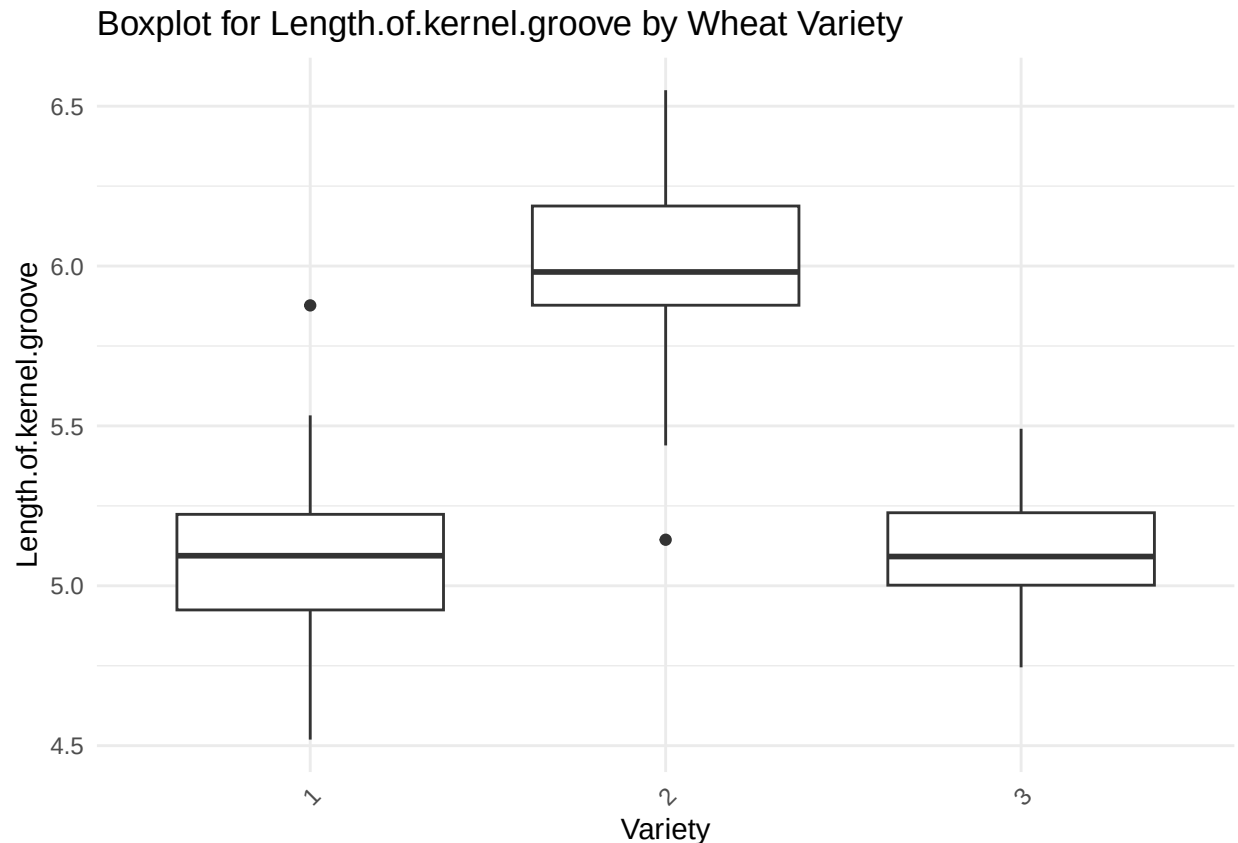










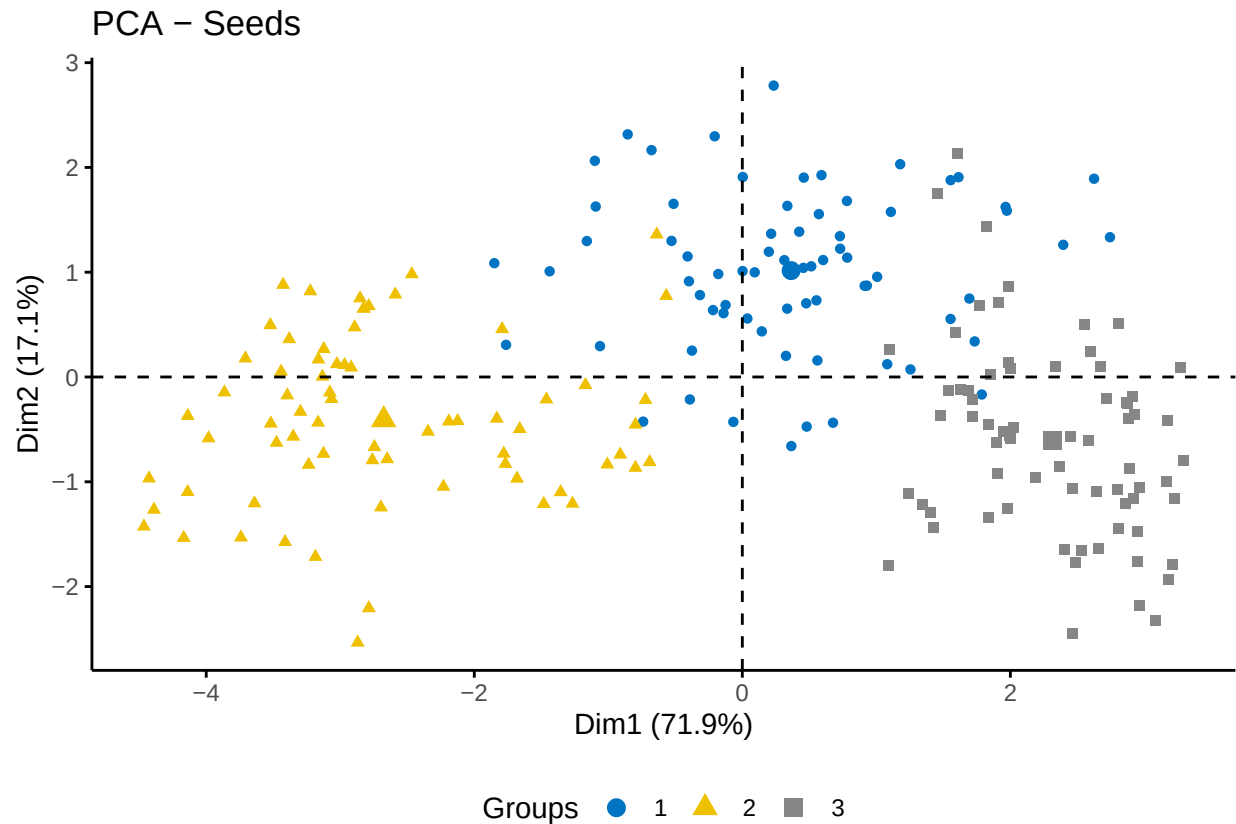


According to the boxplot, we can see that distribution of variables V1, V2, V4, V5 and V6 among Wheat variety is clearly separated. This suggests that there might be significant differences in the values of these variables across the wheat varieties.

Assessing Cluster Tendency

We visualize the data to assess whether they contains any meaningful clusters. As data contains more than 2 variables we need to reduce dimensionality in order to plot a scatterplot. For that we use PCA to reduce dimensionality of the dataset.

```
scaled_data <- scale(Seeds[1:7])
fviz_pca_ind(prcomp(scaled_data), title = "PCA - Seeds",
  habillage = Seeds$Variety, palette = "jco",
  geom = "point", ggtheme = theme_classic(),
  legend = "bottom")
```



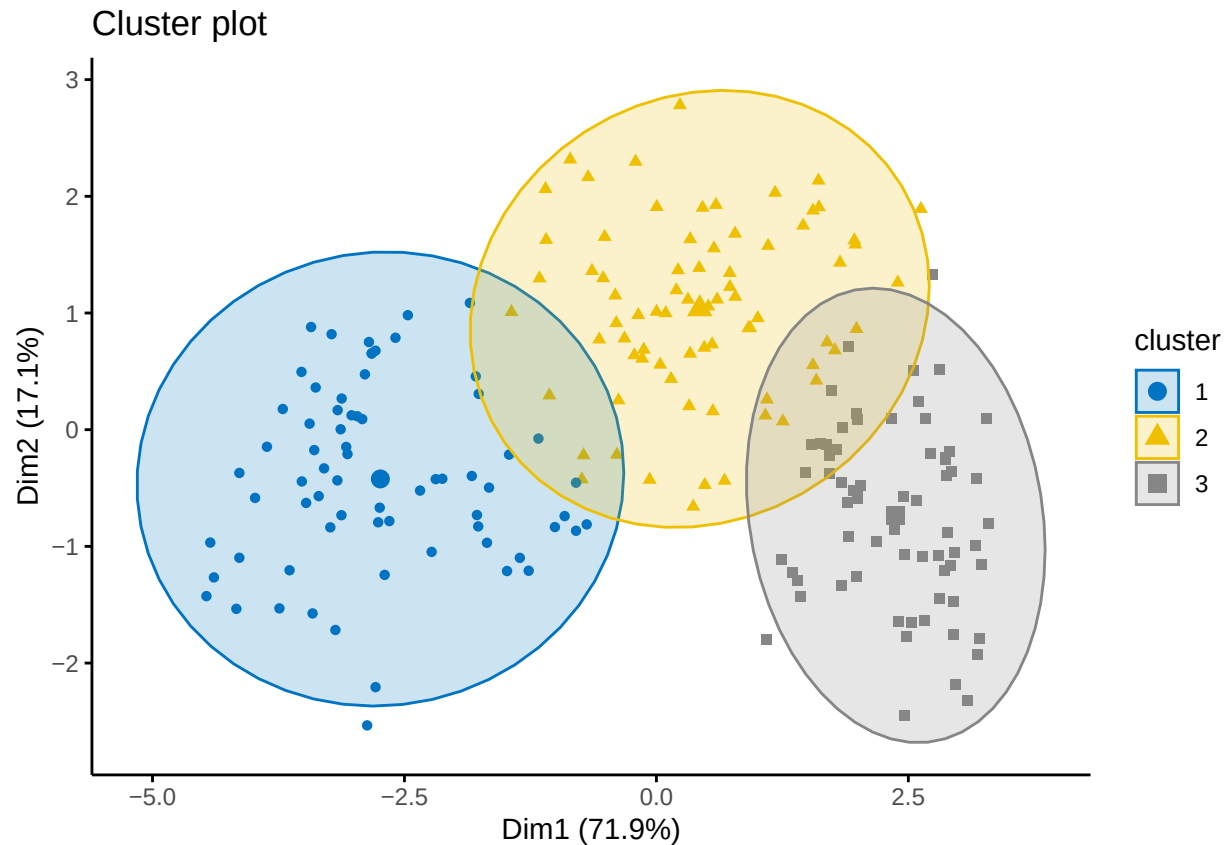
It can be seen that Seeds dataset contains 3 real clusters.

Applying k-means and hierarchical clustering on the dataset

```
set.seed(123)

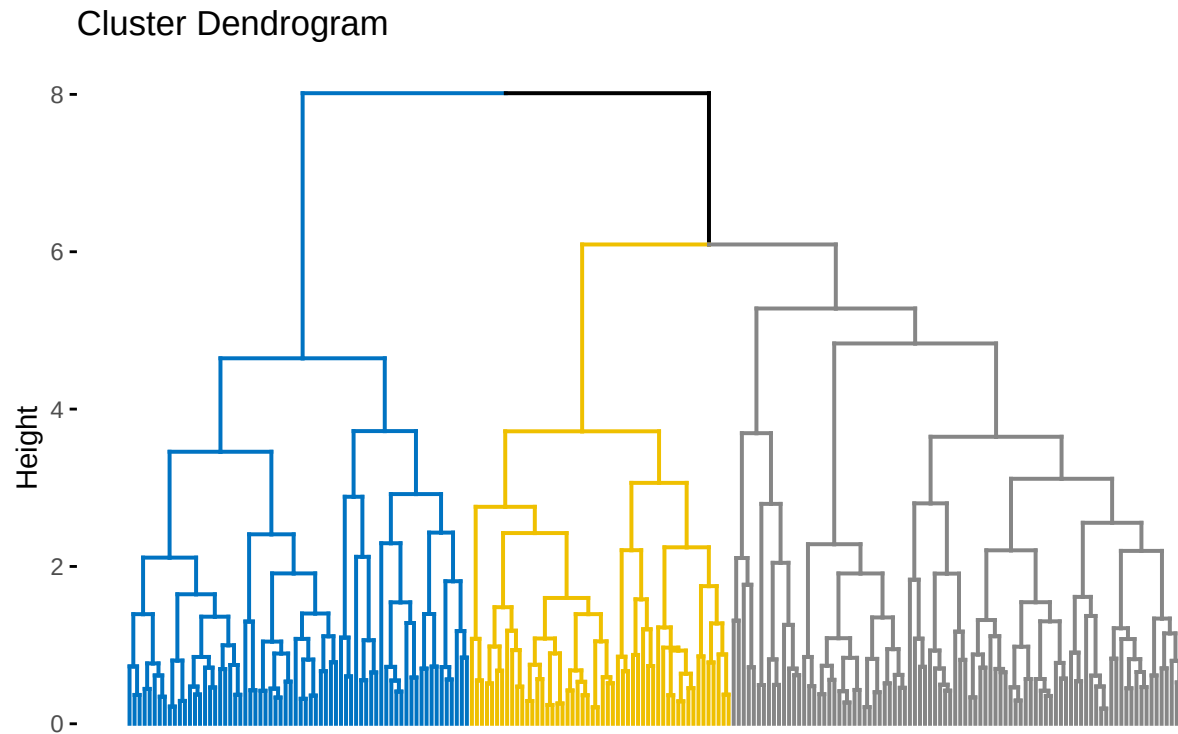
km.res <- kmeans(scaled_data, 3)

fviz_cluster(list(data = scaled_data, cluster = km.res$cluster),
  ellipse.type = "norm", geom = "point", stand = FALSE,
  palette = "jco", ggtheme = theme_classic())
```



```
fviz_dend(hclust(dist(scaled_data)), k = 3, k_colors = "jco",
          as.ggplot = TRUE, show_labels = FALSE)
```

```
## Warning: The '<scale>' argument of 'guides()' cannot be 'FALSE'. Use "none" instead as
## of ggplot2 3.3.4.
## i The deprecated feature was likely used in the factoextra package.
## Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

It can be seen that k-means algorithm and hierarchical clustering impose a 3 distinctive separate clusters in this dataset.

Further to assess if the dataset contains any meaningful clusters we checked for **Hopkins Statistic**

H_0 : There is no meaningful cluster structure in the data. ($H < 0.5$)

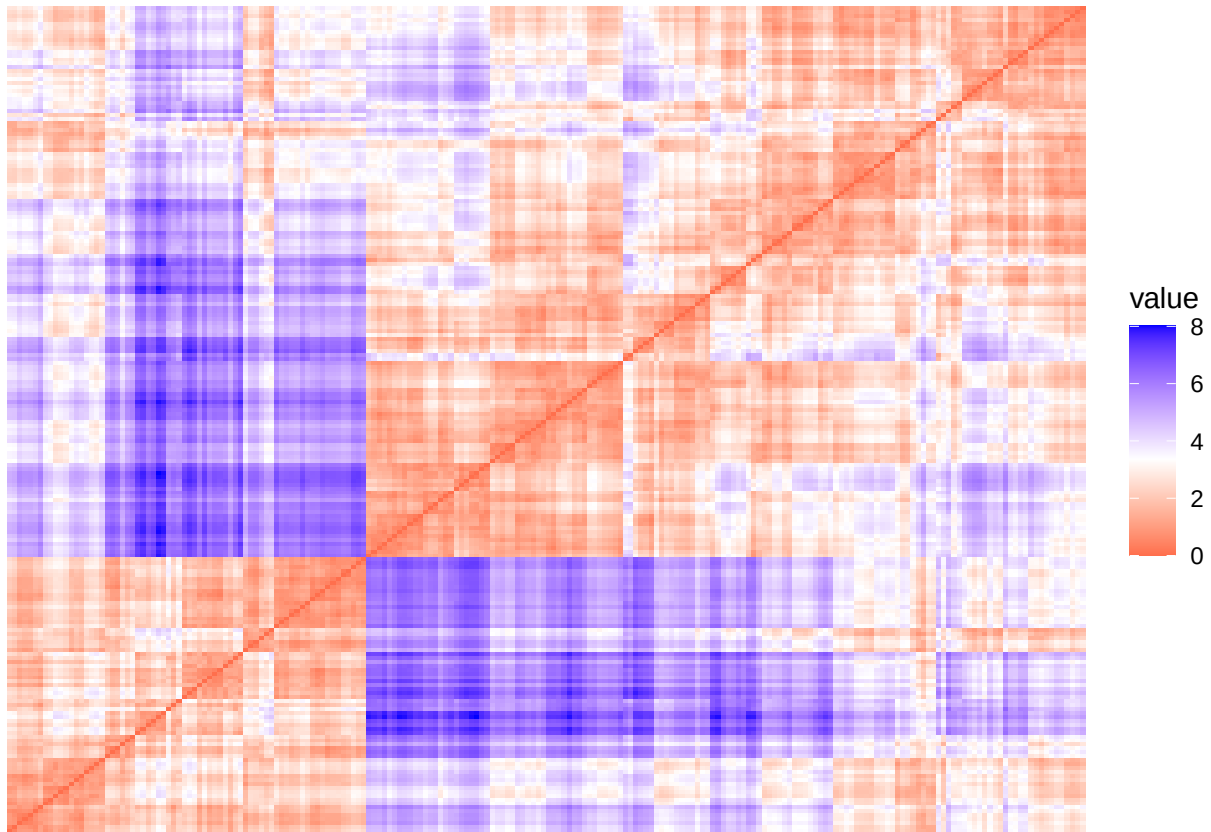
H_a : There is meaningful cluster structure in the data.

```
res1 <- get_clust_tendency(scaled_data, n = nrow(scaled_data)-1, graph = FALSE)
res1$hopkins_stat
```

```
## [1] 0.8027413
```

According to the results, Seeds dataset is highly clusterable (H value = 0.80 which is far above the threshold 0.5)

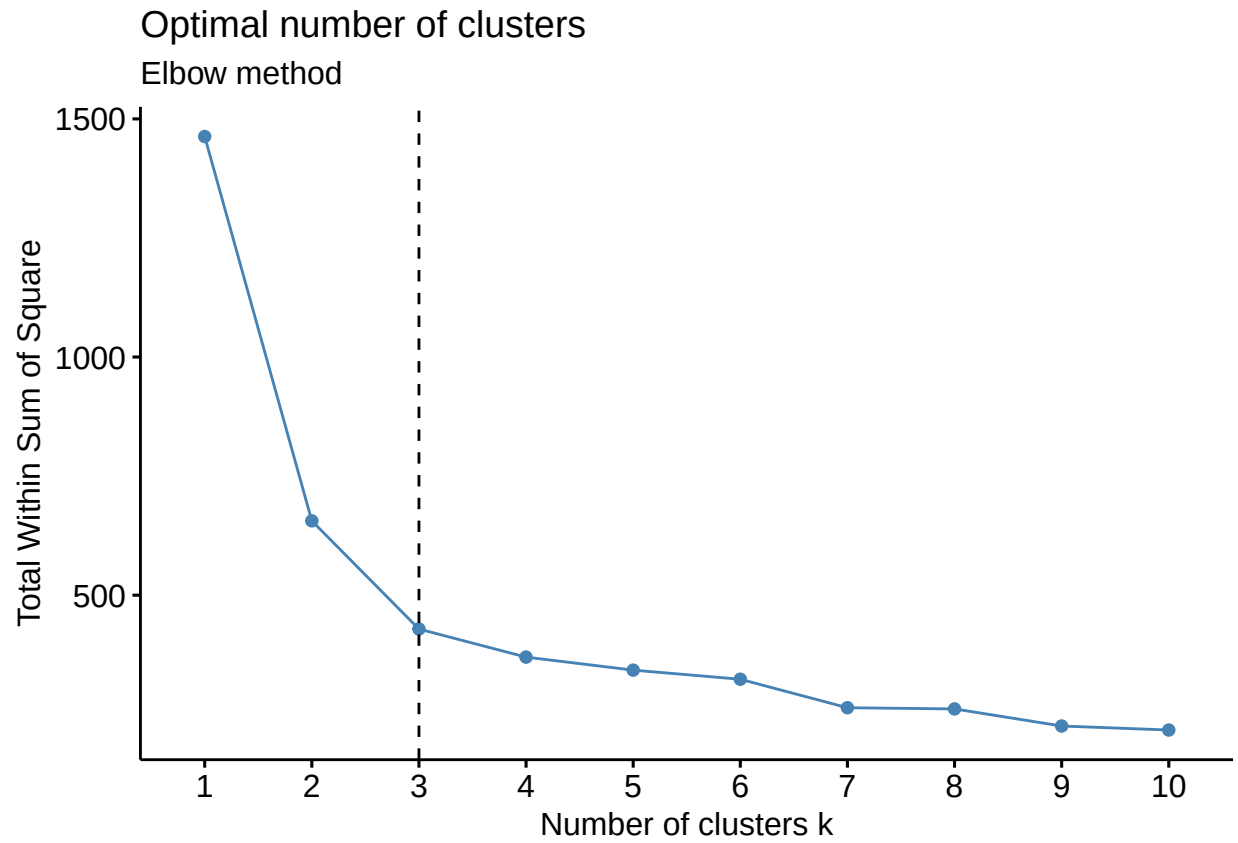
```
fviz_dist(dist(scaled_data), show_labels = FALSE)
```



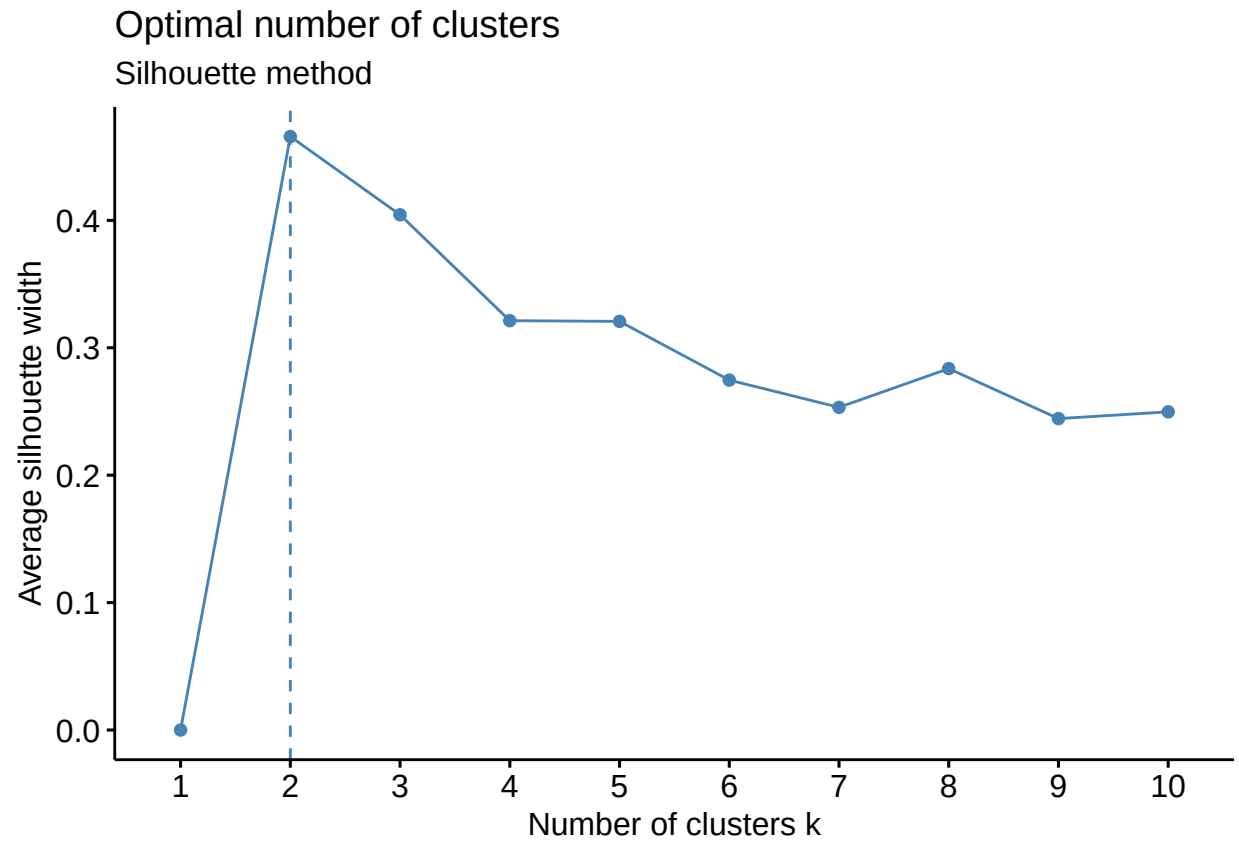
The dissimilarity matrix image shows that there is a cluster structure in the Seeds dataset.

Determining the optimal number of clusters

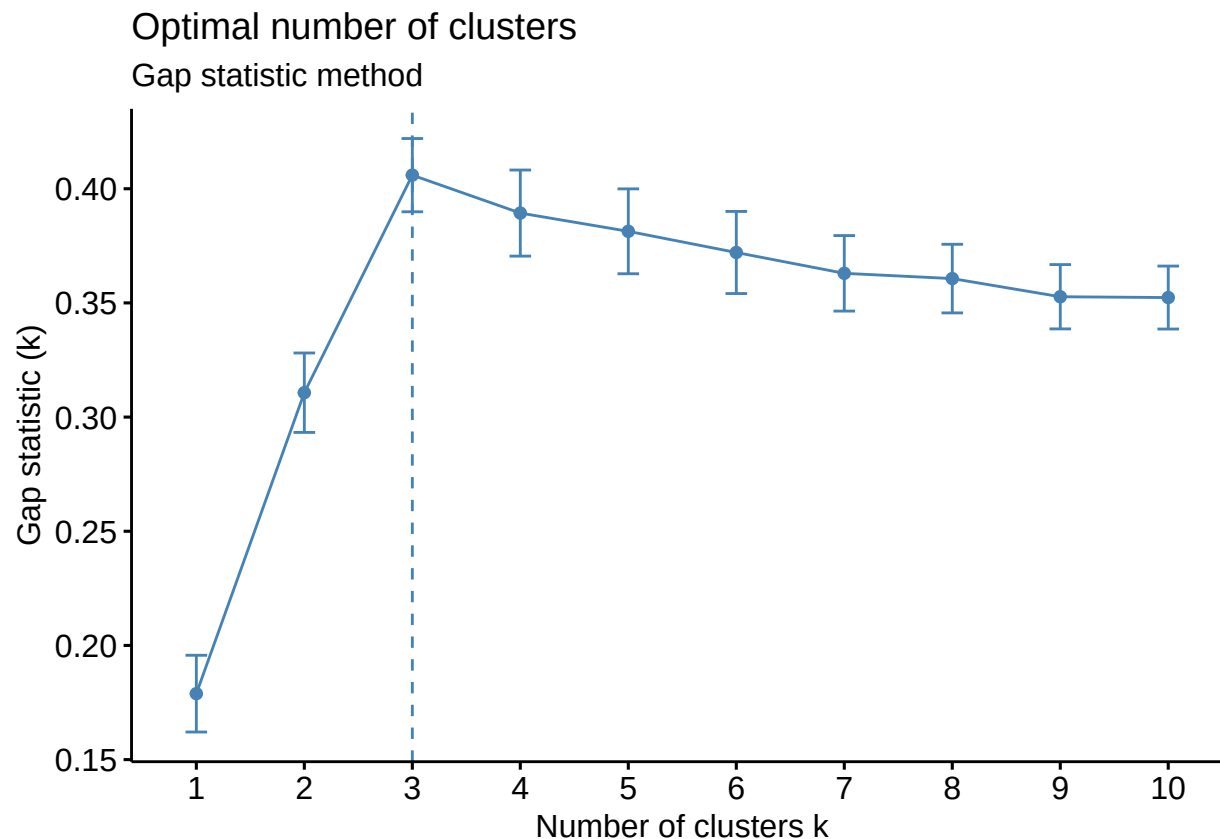
```
# Elbow method  
fviz_nbclust(scaled_data, kmeans, method = "wss") +  
  geom_vline(xintercept = 3, linetype = 2) +  
  labs(subtitle = "Elbow method")
```



```
# Silhouette method  
fviz_nbclust(scaled_data, kmeans, method = "silhouette")+  
  labs(subtitle = "Silhouette method")
```



```
# Gap statistic  
# nboot = 50 to keep the function speedy.  
# recommended value: nboot= 500 for your analysis.  
# Use verbose = FALSE to hide computing progression.  
set.seed(123)  
fviz_nbclust(scaled_data, kmeans, nstart = 25, method = "gap_stat", nboot = 50)+  
  labs(subtitle = "Gap statistic method")
```



- Elbow method - 3 clusters solution suggested
- Silhouette method - 2 clusters solution suggested
- Gap statistic method - 3 clusters solution suggested

According to these observations, it's possible to define $k=3$ as the optimal number of clusters in the Seeds dataset.

K-means

```
set.seed(123)
km.res <- kmeans(scaled_data, 3, nstart = 25)
print(km.res)
```

```
## K-means clustering with 3 clusters of sizes 71, 67, 72
##
## Cluster means:
##      Area Perimeter Compactness Length.of.kernel Width.of.kernel
## 1 -0.1407831 -0.1696372  0.4485346   -0.2571999    0.001643014
## 2  1.2536860  1.2589580  0.5591283    1.2349319    1.162075101
## 3 -1.0277967 -1.0042491 -0.9626050   -0.8955451   -1.082995635
##  Asymmetry.coefficient Length.of.kernel.groove
## 1          -0.66034079              -0.5844965
```

```
## 2          -0.04511157          1.2892273
## 3          0.69314821          -0.6233191
##
## Clustering vector:
## [1] 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 1 3 1 1 1 1 1 3 2 2 2 2
## [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 1 2 2 1 2 1 1 2 3 3 3 3 3 3 3
## [149] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## [186] 3 3 3 3 3 3 3 3 3 3 3 3 3 1 3 1 3 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
##
## Within cluster sum of squares by cluster:
## [1] 144.4586 139.5542 144.5954
## (between_SS / total_SS = 70.7 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"       "
```

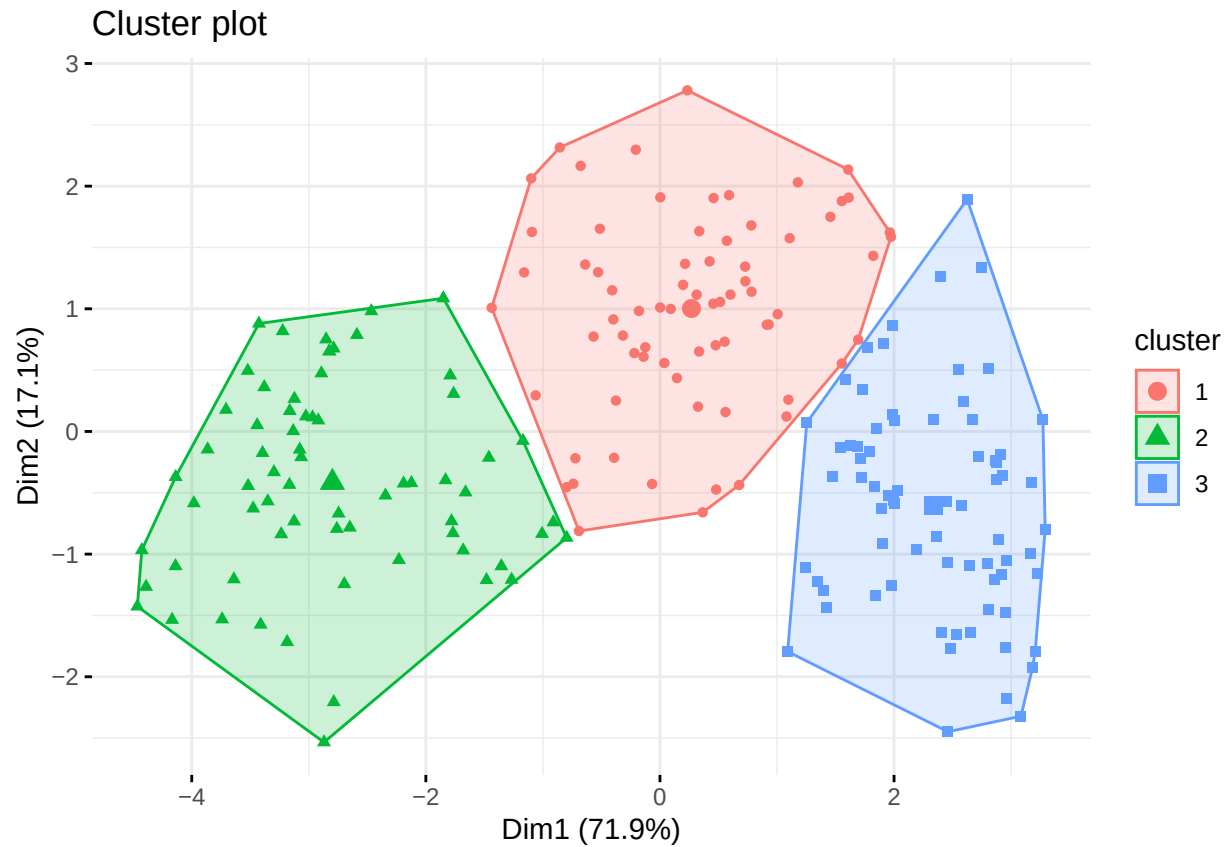
```
aggregate(Seeds, by=list(cluster=km.res$cluster), mean)
```

```
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
## Warning in mean.default(X[[i]], ...): argument is not numeric or logical:
## returning NA
```

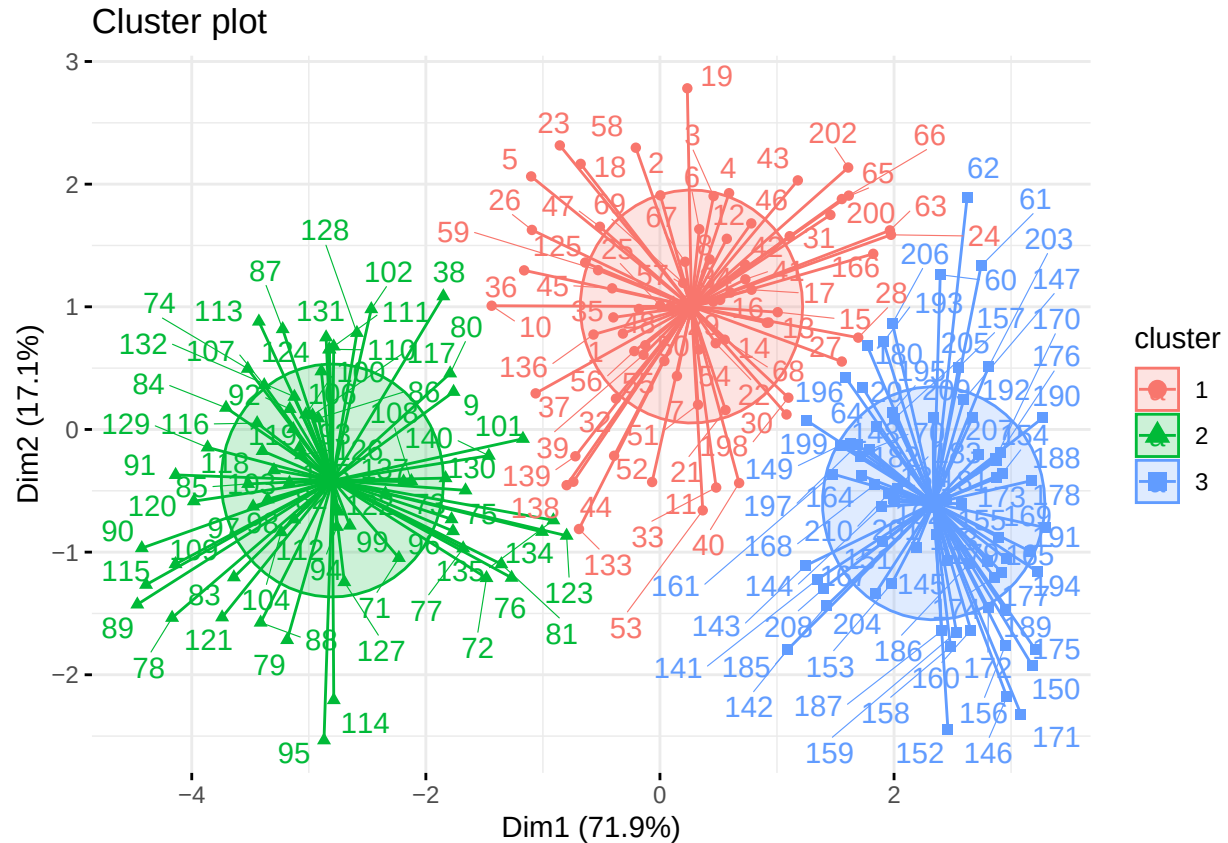
```
## cluster      Area Perimeter Compactness Length.of.kernel Width.of.kernel
## 1           1 14.43789 14.33775 0.8815972          5.514577          3.259225
## 2           2 18.49537 16.20343 0.8842104          6.175687          3.697537
## 3           3 11.85694 13.24778 0.8482528          5.231750          2.849542
## Asymmetry.coefficient Length.of.kernel.groove Variety
## 1              2.707341              5.120803      NA
## 2              3.632373              6.041701      NA
## 3              4.742389              5.101722      NA
```

Visualizing k-means clusters

```
fviz_cluster(km.res, data = scaled_data, geom = "point", ellipse.type = "convex", ggtheme = theme_minimal)
```



```
fviz_cluster(km.res, data = scaled_data,  
  ellipse.type = "euclid", # Concentration ellipse  
  star.plot = TRUE, # Add segments from centroid to items  
  repel = TRUE, # Avoid label overplotting (slow)  
  ggtheme = theme_minimal()  
)
```



PAM

```
pam.res <- pam(scaled_data, 3)
print(pam.res)
```

```
## Medoids:
##      ID      Area Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,]  49 -0.01976967 -0.0300819  0.4613499   -0.1885358    0.08576648
## [2,] 105  1.40993126  1.4247880  0.5036700    1.3981443    1.31420772
## [3,] 163 -0.96144769 -0.8800322 -1.2441514   -0.8159854   -1.08972470
##      Asymmetry.coefficient Length.of.kernel.groove
## [1,]          -0.6625628          -0.6044419
## [2,]          -0.2209434           1.5055095
## [3,]           0.8565016          -0.7366954
## Clustering vector:
##  [1] 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 1 1 3 1 1 1 1 1 3 2 2 2 2
## [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 2 2 2 2 2 2 1 1 1 2 1 1 2 3 3 3 3 3 3 3
## [149] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## [186] 3 3 3 3 3 3 3 3 3 3 3 3 1 3 1 3 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## Objective function:
##      build      swap
## 1.388781 1.354058
```



```
##
## Available components:
## [1] "medoids"      "id.med"      "clustering"  "objective"   "isolation"
## [6] "clusinfo"     "silinfo"     "diss"        "call"        "data"
```

```
dd <- cbind(scaled_data, cluster = pam.res$cluster)
head(dd, n = 3)
```

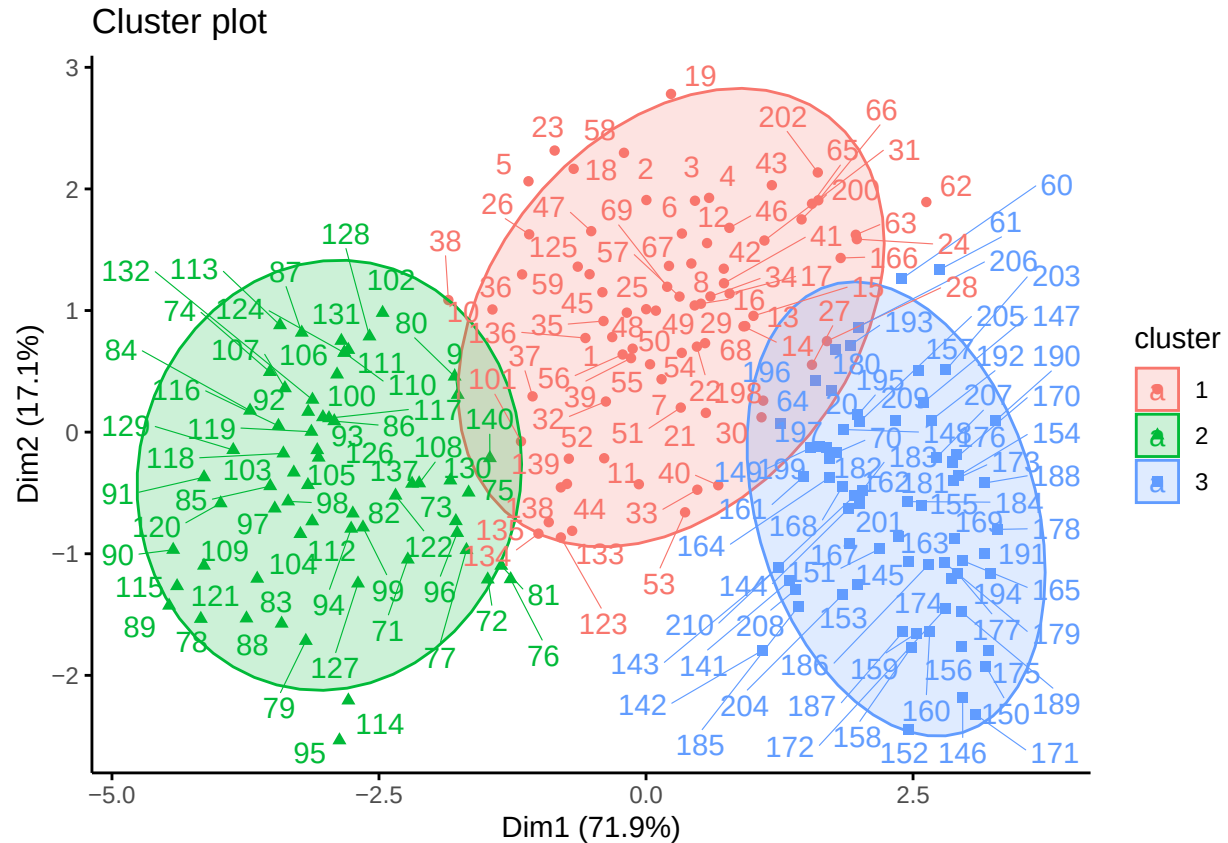
```
##           Area      Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,]  0.14175904  0.214948819 6.045733e-05      0.3034930      0.1413640
## [2,]  0.01116136  0.008204153 4.274938e-01     -0.1682227      0.1969616
## [3,] -0.19160873 -0.359341919 1.438945e+00     -0.7618171      0.2075516
##      Asymmetry.coefficient Length.of.kernel.groove cluster
## [1,]          -0.9838010             -0.3826631         1
## [2,]          -1.7839036             -0.9198156         1
## [3,]          -0.6658882             -1.1863572         1
```

```
pam.res$medoids
```

```
##           Area      Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,] -0.01976967 -0.0300819  0.4613499      -0.1885358      0.08576648
## [2,]  1.40993126  1.4247880  0.5036700       1.3981443      1.31420772
## [3,] -0.96144769 -0.8800322 -1.2441514      -0.8159854     -1.08972470
##      Asymmetry.coefficient Length.of.kernel.groove
## [1,]          -0.6625628             -0.6044419
## [2,]          -0.2209434              1.5055095
## [3,]           0.8565016             -0.7366954
```

Visualizing PAM result

```
fviz_cluster(pam.res,
  ellipse.type = "t", # Concentration ellipse
  repel = TRUE, # Avoid label overplotting (slow)
  ggtheme = theme_classic()
)
```



CLARA

```
clara.res <- clara(scaled_data, 3, samples = 50, pamLike = TRUE)
print(clara.res)
```

```
## Call:      clara(x = scaled_data, k = 3, samples = 50, pamLike = TRUE)
## Medoids:
##           Area Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,] -0.01976967 -0.0300819  0.4613499   -0.1885358    0.08576648
## [2,]  1.40993126  1.4247880  0.5036700    1.3981443    1.31420772
## [3,] -0.91333276 -0.9183182 -0.5543333   -0.8769248   -0.92557954
##           Asymmetry.coefficient Length.of.kernel.groove
## [1,]                -0.6625628                -0.6044419
## [2,]                -0.2209434                 1.5055095
## [3,]                 0.7693749                 -0.5088125
## Objective function:  1.356174
## Clustering vector:   int [1:210] 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 ...
## Cluster sizes:      71 62 77
## Best sample:
## [1]  6  8 10 15 24 30 39 41 49 54 59 63 70 71 76 86 93 98 100
## [20] 105 111 112 114 120 121 124 125 128 129 131 133 135 138 144 145 146 153 160
## [39] 176 180 182 183 188 195 201 206
##
## Available components:
```

```
## [1] "sample"      "medoids"      "i.med"       "clustering"  "objective"
## [6] "clusinfo"     "diss"         "call"        "silinfo"     "data"
```

```
dd <- cbind(scaled_data, cluster = clara.res$cluster)
head(dd, n = 4)
```

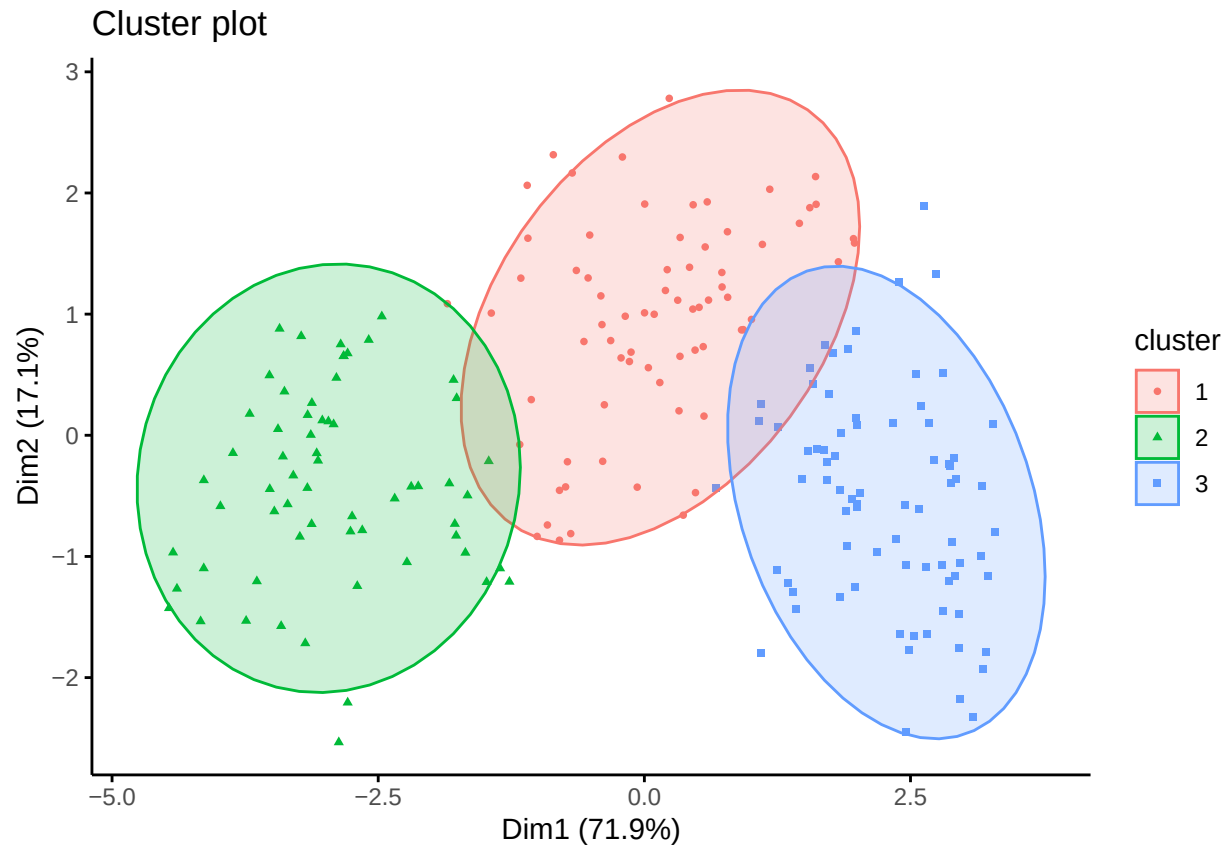
```
##           Area      Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,]  0.14175904  0.214948819 6.045733e-05      0.3034930      0.1413640
## [2,]  0.01116136  0.008204153 4.274938e-01     -0.1682227      0.1969616
## [3,] -0.19160873 -0.359341919 1.438945e+00     -0.7618171      0.2075516
## [4,] -0.34626388 -0.474200066 1.036904e+00     -0.6873357      0.3187467
##      Asymmetry.coefficient Length.of.kernel.groove cluster
## [1,]          -0.9838010              -0.3826631      1
## [2,]          -1.7839036              -0.9198156      1
## [3,]          -0.6658882              -1.1863572      1
## [4,]          -0.9585276              -1.2270506      1
```

```
# Medoids
clara.res$medoids
```

```
##           Area      Perimeter Compactness Length.of.kernel Width.of.kernel
## [1,] -0.01976967 -0.0300819  0.4613499     -0.1885358      0.08576648
## [2,]  1.40993126  1.4247880  0.5036700      1.3981443      1.31420772
## [3,] -0.91333276 -0.9183182 -0.5543333     -0.8769248     -0.92557954
##      Asymmetry.coefficient Length.of.kernel.groove
## [1,]          -0.6625628              -0.6044419
## [2,]          -0.2209434              1.5055095
## [3,]           0.7693749              -0.5088125
```

Visualizing CLARA clusters

```
fviz_cluster(clara.res,
  ellipse.type = "t", # Concentration ellipse
  geom = "point", pointsize = 1,
  ggtheme = theme_classic()
)
```



Cluster Validation Statistics

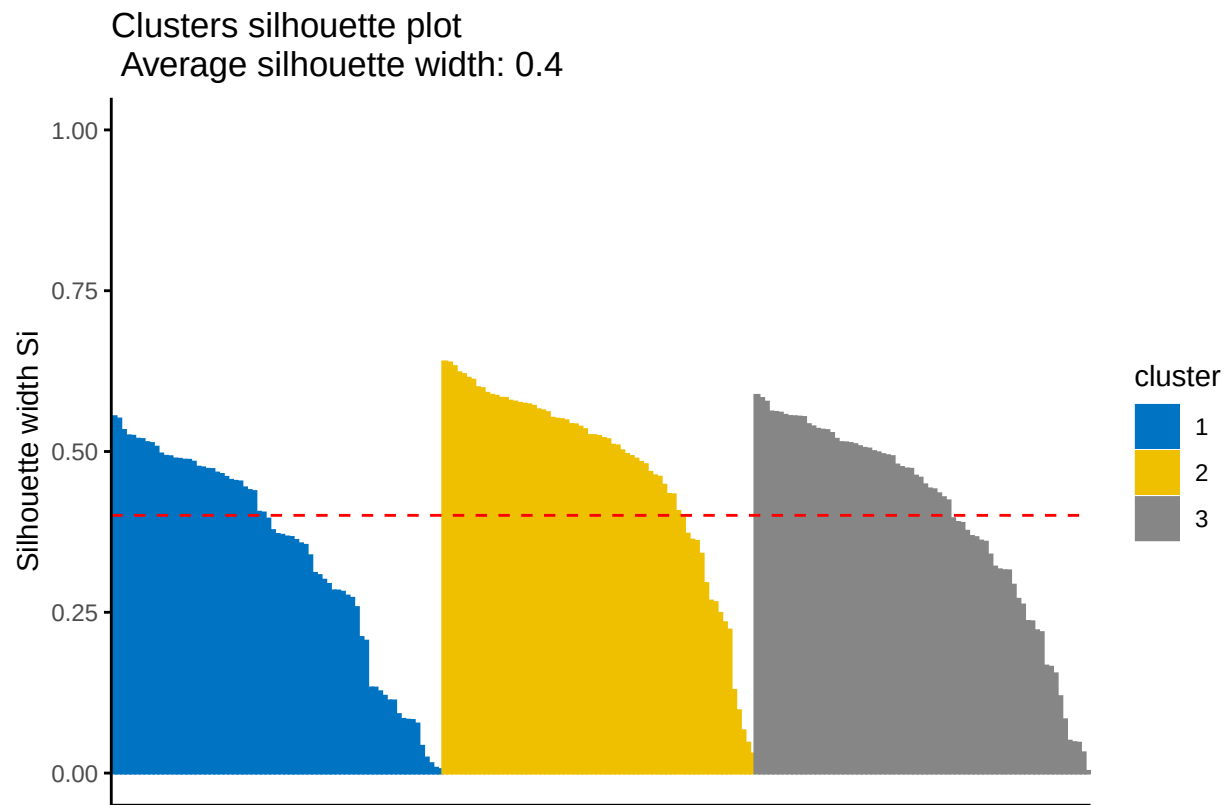
Internal cluster validation - Silhouette, Dunn Index, Connectivity

Silhouette coefficient

```
sil_kmeans <- silhouette(km.res$cluster, dist(scaled_data))
sil_pam <- silhouette(pam.res$cluster, dist(scaled_data))
sil_clara <- silhouette(clara.res$cluster, dist(scaled_data))

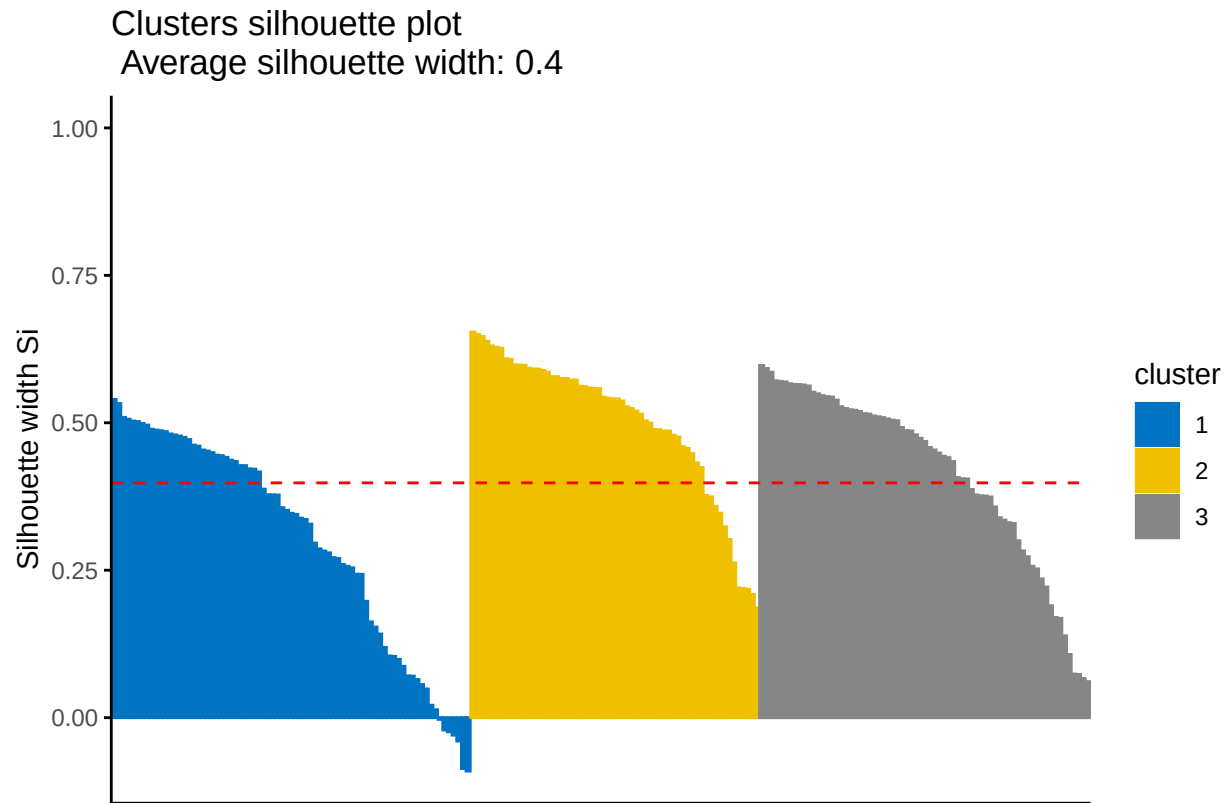
fviz_silhouette(sil_kmeans, palette = "jco", ggtheme = theme_classic())
```

```
##   cluster size ave.sil.width
## 1      1   71         0.34
## 2      2   67         0.47
## 3      3   72         0.40
```



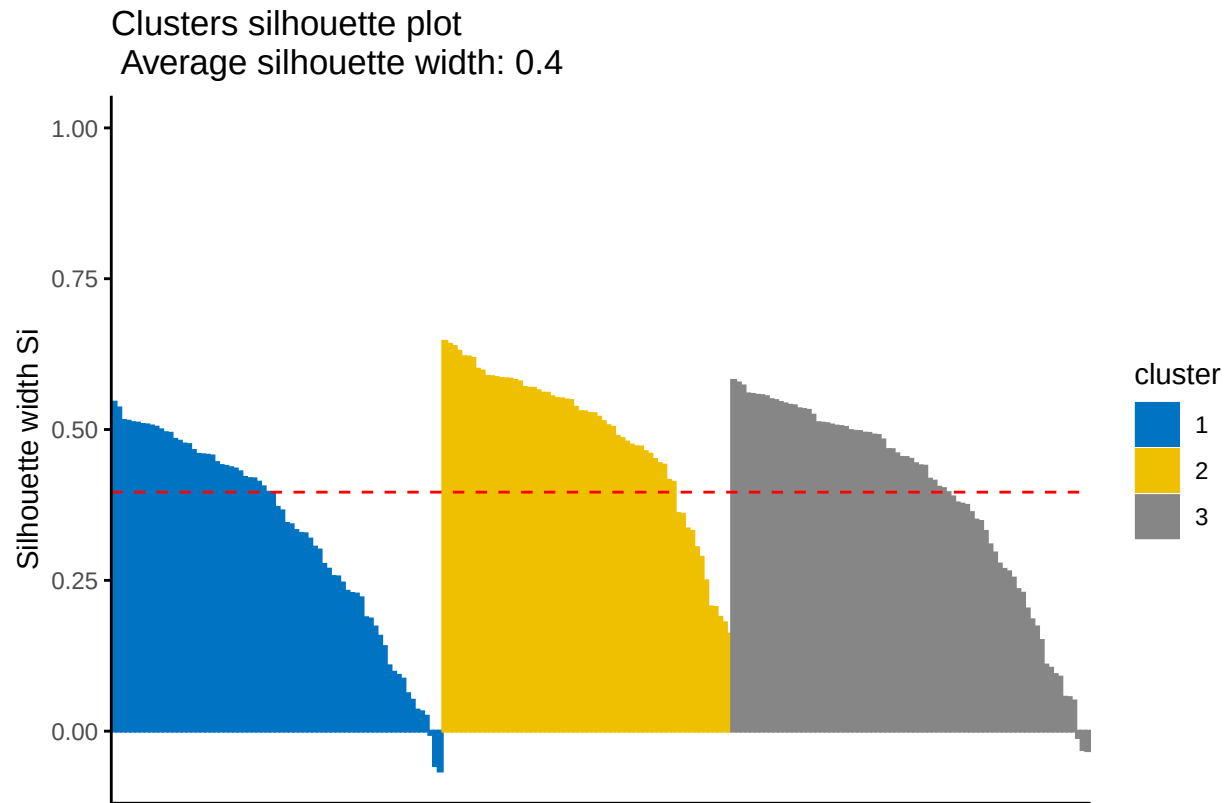
```
fviz_silhouette(sil_pam, palette = "jco", ggtheme = theme_classic())
```

##	cluster	size	ave.sil.width
## 1	1	77	0.30
## 2	2	62	0.50
## 3	3	71	0.42



```
fviz_silhouette(sil_clara, palette = "jco", ggtheme = theme_classic())
```

##	cluster	size	ave.sil.width
## 1	1	71	0.33
## 2	2	62	0.49
## 3	3	77	0.39



Dunn Index

```
# Statistics for k-means clustering
km_stats <- cluster.stats(dist(scaled_data), km.res$cluster)
# Dun index
km_stats$dunn
```

```
## [1] 0.1188182
```

```
pam_stats <- cluster.stats(dist(scaled_data), pam.res$cluster)
pam_stats$dunn
```

```
## [1] 0.1114771
```

```
clara_stats <- cluster.stats(dist(scaled_data), clara.res$cluster)
clara_stats$dunn
```

```
## [1] 0.1114771
```

Connectivity

```
connectivity(dist(scaled_data), km.res$cluster, neighbSize = 2)
```

```
## [1] 17
```

```
connectivity(dist(scaled_data), pam.res$cluster, neighbSize = 2)
```

```
## [1] 18.5
```

```
connectivity(dist(scaled_data), clara.res$cluster, neighbSize = 2)
```

```
## [1] 13
```

External cluster validation - Rand Index, Adjuster Rand Index

Rand Index

```
table(Seeds$Variety, km.res$cluster)
```

```
##
##      1  2  3
##  1 62  2  6
##  2  5 65  0
##  3  4  0 66
```

```
variety <- as.numeric(Seeds$Variety)
clust_stats <- cluster.stats(d = dist(scaled_data), variety, km.res$cluster)
# Corrected Rand index
clust_stats$corrected.rand
```

```
## [1] 0.7732937
```

```
# VI
clust_stats$vi
```

```
## [1] 0.5978132
```

The corrected Rand index provides a measure for assessing the similarity between two partitions, adjusted for chance. Agreement between the seed variety types and the cluster solution is 0.77 using Rand index and 0.59 using Meila's VI.

```
table(Seeds$Variety, pam.res$cluster)
```

```
##
##      1  2  3
##  1 64  1  5
##  2  9 61  0
##  3  4  0 66
```



```
cluster.stats(d = dist(scaled_data), variety, pam.res$cluster)$corrected.rand
```

```
## [1] 0.7469561
```

```
table(Seeds$Variety, clara.res$cluster)
```

```
##  
##      1  2  3  
##  1 59  1 10  
##  2  9 61  0  
##  3  3  0 67
```

```
cluster.stats(d = dist(scaled_data), variety, clara.res$cluster)$corrected.rand
```

```
## [1] 0.7043879
```

PRINCIPAL COMPONENT ANALYSIS

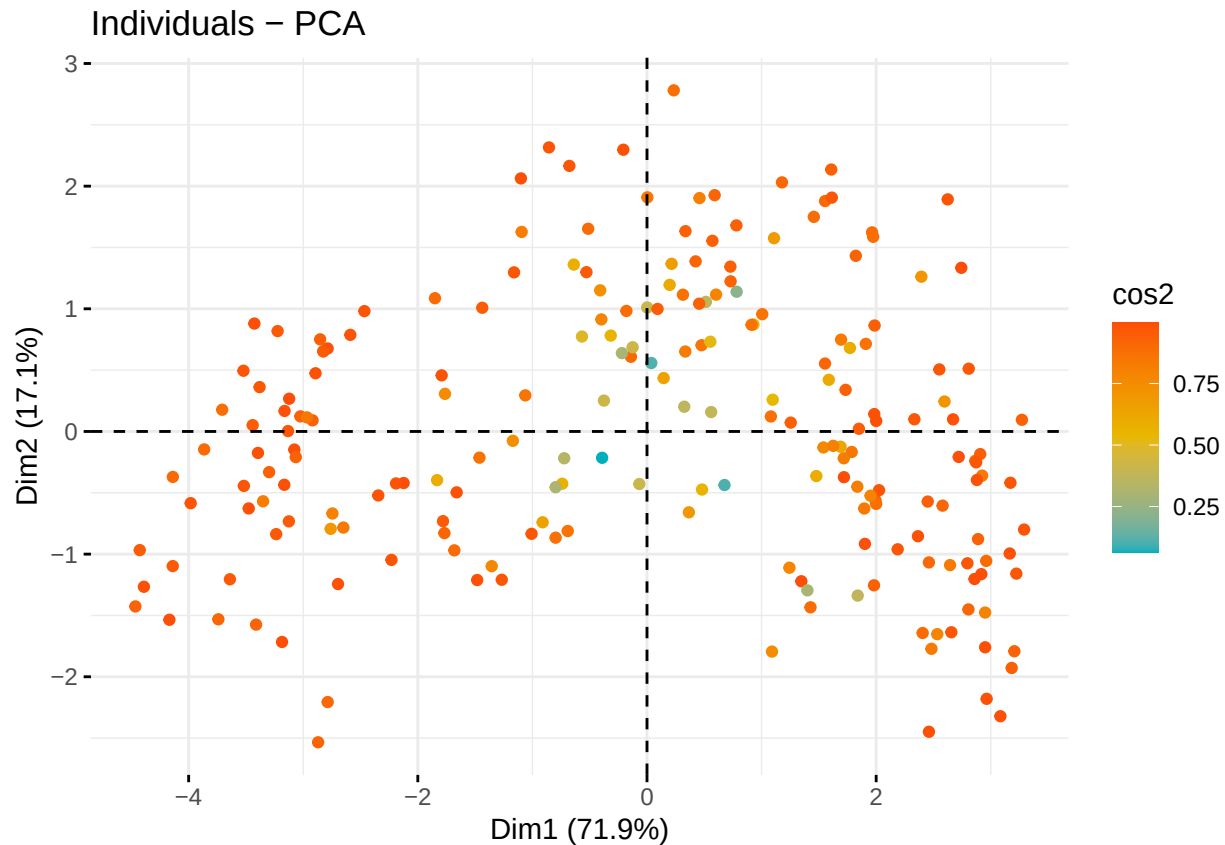
We are applying PCA to the SEEDS dataset before clustering to reduce the dimensionality while retaining most of the variance in the data. This step simplifies the dataset, reduces noise, and enhances computational efficiency. By focusing on uncorrelated components, PCA helps ensure that the clustering algorithms (K-means, K-medoids, and K-means++) operate on a more interpretable and balanced feature set, improving the overall clustering performance and stability.

```
#PCA  
pca_result <- prcomp(Seeds[, 1:7], scale. = TRUE)  
summary(pca_result)
```

```
## Importance of components:  
##          PC1      PC2      PC3      PC4      PC5      PC6      PC7  
## Standard deviation  2.2430 1.0943 0.82341 0.26147 0.13680 0.07302 0.02850  
## Proportion of Variance 0.7187 0.1711 0.09686 0.00977 0.00267 0.00076 0.00012  
## Cumulative Proportion 0.7187 0.8898 0.98668 0.99645 0.99912 0.99988 1.00000
```

The first principal component PC1 explains approximately 71.87% of the variance in the SEEDS dataset, while the second PC2 and third PC3 components contribute 17.11% and 9.69%, respectively. Together, the first three components account for nearly 98.67% of the total variance, indicating that these components effectively capture the majority of the information within the data. Therefore, using the first few principal components for clustering can significantly simplify the dataset while retaining its essential structure.

```
# Plot PCA results  
fviz_pca_ind(pca_result, geom.ind = "point", col.ind = "cos2", gradient.cols = c("#00AFBB", "#E7B800",
```



#SCALING

Before applying K-means clustering, we will scale the data to ensure consistent feature contributions. The K-means algorithm relies on Euclidean distance, meaning features with larger scales can dominate clustering results. In the SEEDS dataset, attributes such as Area, Perimeter, and Length of kernel groove vary significantly compared to features like Compactness or Asymmetry coefficient. By scaling, we balance these differences, allowing each feature to equally influence cluster formation and improve clustering stability across K-means, K-medoids, and K-means++ methods.

```
scaled_data <- scale(Seeds[, 1:7])
```

We assume three initial clusters for the K-means, K-means++, and K-medoids clustering algorithms based on the known structure of the SEEDS dataset, which consists of three distinct wheat varieties: Kama, Rosa, and Canadian.

#K-MEANS

```
# k-means 3 clusters
set.seed(123)
kmeans_result <- kmeans(scaled_data, centers = 3, nstart = 25)
print(kmeans_result)
```

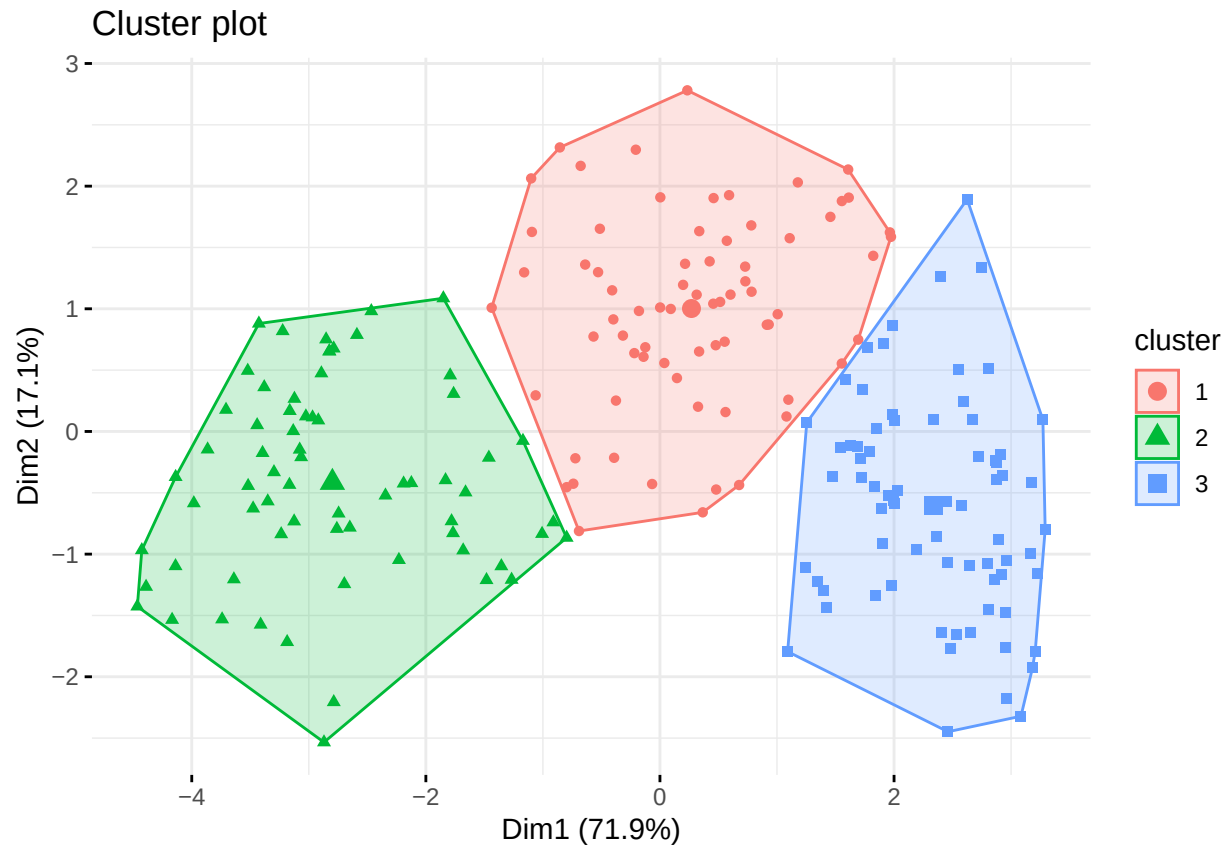
```
## K-means clustering with 3 clusters of sizes 71, 67, 72
##
## Cluster means:
```

```
##           Area  Perimeter Compactness Length.of.kernel Width.of.kernel
## 1 -0.1407831 -0.1696372  0.4485346      -0.2571999      0.001643014
## 2  1.2536860  1.2589580  0.5591283       1.2349319      1.162075101
## 3 -1.0277967 -1.0042491 -0.9626050      -0.8955451     -1.082995635
##   Asymmetry.coefficient Length.of.kernel.groove
## 1          -0.66034079          -0.5844965
## 2          -0.04511157           1.2892273
## 3           0.69314821          -0.6233191
##
## Clustering vector:
##  [1] 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 1 3 1 1 1 1 1 3 2 2 2 2
## [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 1 2 2 1 2 1 1 2 3 3 3 3 3 3 3 3
## [149] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## [186] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 1 3 1 3 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
##
## Within cluster sum of squares by cluster:
## [1] 144.4586 139.5542 144.5954
## (between_SS / total_SS =  70.7 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

The K-means algorithm partitioned the SEEDS dataset into three clusters with sizes of 71, 67, and 72, which aligns with the dataset's known three wheat varieties. The cluster centers reveal the mean standardized values for each feature within the clusters, highlighting differences across attributes such as Area, Perimeter, and Length of kernel groove.

The within-cluster sum of squares for each cluster (144.46, 139.55, and 144.60) indicates the compactness of the clusters, with a between-cluster sum of squares to total sum of squares ratio of 70.7%, suggesting a moderately good cluster separation. This result implies that the clusters formed capture a significant portion of the dataset's variance but may benefit from further optimization or comparison with other clustering methods.

```
# Visualize the clusters
fviz_cluster(kmeans_result, data = scaled_data, geom = "point", ellipse.type = "convex", ggtheme = theme_
```



```
# KMEANS cluster centers
kmeans_result$centers
```

```
##          Area  Perimeter Compactness Length.of.kernel Width.of.kernel
## 1 -0.1407831 -0.1696372  0.4485346      -0.2571999      0.001643014
## 2  1.2536860  1.2589580  0.5591283       1.2349319      1.162075101
## 3 -1.0277967 -1.0042491 -0.9626050      -0.8955451     -1.082995635
##  Asymmetry.coefficient Length.of.kernel.groove
## 1          -0.66034079             -0.5844965
## 2          -0.04511157              1.2892273
## 3           0.69314821             -0.6233191
```

```
#K-MEANS++
```

```
#K-means++
set.seed(123)
kmeans_plus_result <- kmeans(scaled_data, centers = 3, nstart = 25, algorithm = "Lloyd")
```

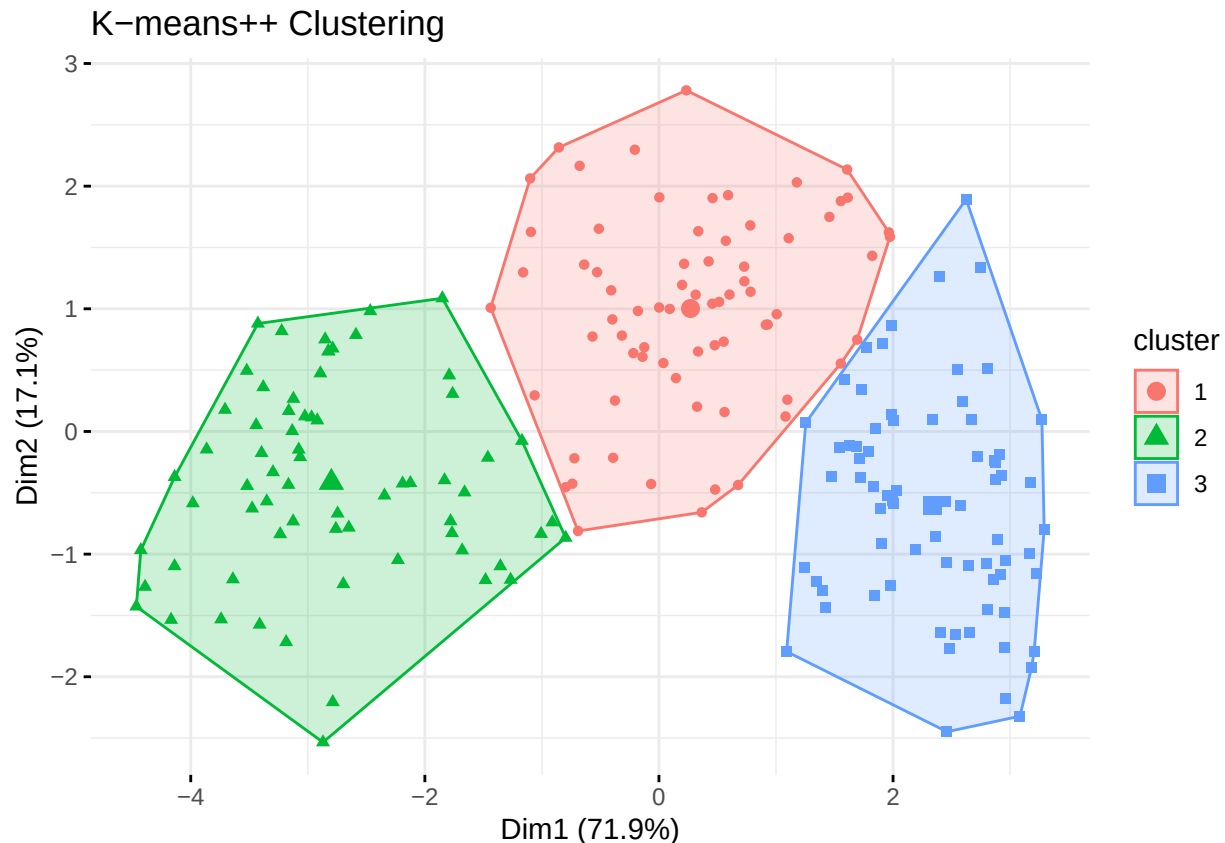
```
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
```

```
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
```

```
print(kmeans_plus_result)
```

```
## K-means clustering with 3 clusters of sizes 71, 67, 72
##
## Cluster means:
##      Area Perimeter Compactness Length.of.kernel Width.of.kernel
## 1 -0.1407831 -0.1696372  0.4485346      -0.2571999    0.001643014
## 2  1.2536860  1.2589580  0.5591283       1.2349319    1.162075101
## 3 -1.0277967 -1.0042491 -0.9626050      -0.8955451   -1.082995635
##  Asymmetry.coefficient Length.of.kernel.groove
## 1          -0.66034079          -0.5844965
## 2          -0.04511157           1.2892273
## 3           0.69314821          -0.6233191
##
## Clustering vector:
##  [1] 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 1 3 1 1 1 1 1 3 2 2 2 2
## [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [149] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## [186] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
##
## Within cluster sum of squares by cluster:
## [1] 144.4586 139.5542 144.5954
## (between_SS / total_SS =  70.7 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

```
fviz_cluster(kmeans_plus_result, data = scaled_data, geom = "point", ellipse.type = "convex", ggtheme =
```



The K-means++ algorithm generated three clusters with sizes as indicated in the result. The between-cluster sum of squares to total sum of squares ratio is 70.7%, indicating the proportion of variance explained by the clusters. This value shows moderate clustering quality, suggesting that the data has distinguishable groups, though not perfectly separated.

As it was not clear for me the difference between K-means and K-means++ here there are some differences. Not sure if we sure include this or not having into account that he did not teach about K-means++

Difference Between K-means and K-means++:

Initialization: The primary difference lies in the initialization step. K-means++ selects initial centroids in a more intelligent way, ensuring that they are spread out, which helps avoid poor cluster initialization and local minima.

Stability and Efficiency: K-means++ often leads to better and more stable results compared to traditional K-means, as it reduces the chances of random initialization leading to suboptimal clustering.

Clustering Outcome: K-means++ typically yields clusters with lower within-cluster variance and improves convergence speed due to better initial centroids.

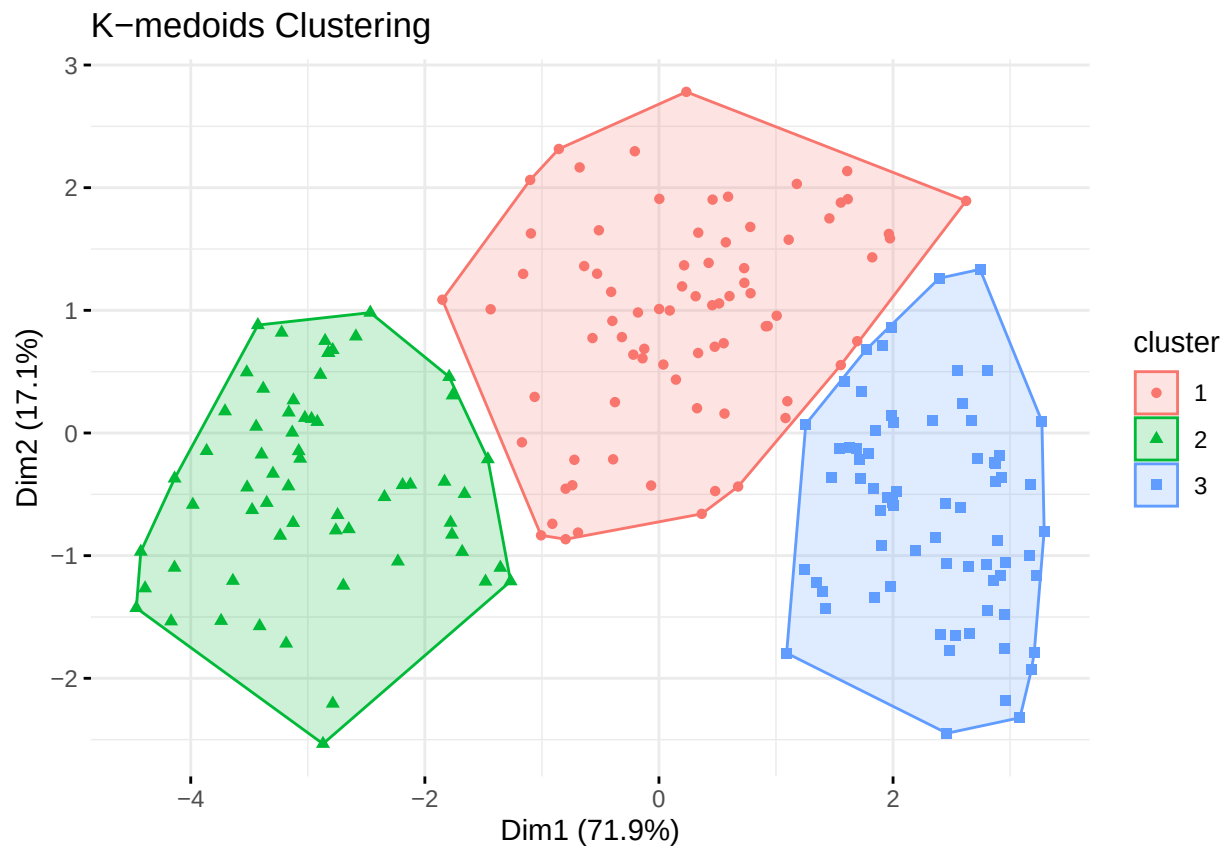
#K-MEDOIDS

```
# K-medoids clustering
pam_result <- pam(scaled_data, k = 3)
print(pam_result)
```

```
## Medoids:
##      ID      Area  Perimeter Compactness Length.of.kernel Width.of.kernel
```

```
## [1,] 49 -0.01976967 -0.0300819 0.4613499 -0.1885358 0.08576648
## [2,] 105 1.40993126 1.4247880 0.5036700 1.3981443 1.31420772
## [3,] 163 -0.96144769 -0.8800322 -1.2441514 -0.8159854 -1.08972470
##      Asymmetry.coefficient Length.of.kernel.groove
## [1,]          -0.6625628          -0.6044419
## [2,]          -0.2209434           1.5055095
## [3,]           0.8565016          -0.7366954
## Clustering vector:
## [1] 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 1 1 3 1 1 1 1 1 3 2 2 2 2
## [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 2 2 2 2 2 2 1 1 1 1 2 1 1 2 3 3 3 3 3 3
## [149] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## [186] 3 3 3 3 3 3 3 3 3 3 3 3 3 1 3 1 3 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
## Objective function:
##      build      swap
## 1.388781 1.354058
##
## Available components:
## [1] "medoids"      "id.med"      "clustering"  "objective"   "isolation"
## [6] "clusinfo"     "silinfo"     "diss"        "call"        "data"
```

```
fviz_cluster(pam_result, geom = "point", ellipse.type = "convex", ggtheme = theme_minimal(), main = "K-medoids Clustering")
```



The K-medoids clustering algorithm produced three clusters, represented by the clustering vector with specific assignments. The objective function values (build = 1.388781 and swap = 1.354058) indicate the

cost of forming these clusters, with lower values implying better compactness and overall cluster quality.

###here there are additional notes regarding K-medoids Robustness: K-medoids is more robust to outliers compared to K-means because it uses actual data points (medoids) as cluster centers.

Cluster Size: The distribution of points across the clusters indicates balance, contributing to stability in cluster assignments.

Objective Values: These highlight the cost of forming clusters during the medoid selection (build phase) and optimization (swap phase), suggesting the overall effectiveness of the algorithm.

#CLUSTER STABILITY

```
# Compare clustering stability using silhouette scores
sil_kmeans <- silhouette(kmeans_result$cluster, dist(scaled_data))
mean(sil_kmeans[, 3]) # Average silhouette width for K-means
```

```
## [1] 0.4007271
```

```
sil_kmeans_plus <- silhouette(kmeans_plus_result$cluster, dist(scaled_data))
mean(sil_kmeans_plus[, 3]) # Average silhouette width for K-means++
```

```
## [1] 0.4007271
```

```
sil_pam <- silhouette(pam_result$clustering, dist(scaled_data))
mean(sil_pam[, 3]) # Average silhouette width for K-medoids
```

```
## [1] 0.3981667
```

The average silhouette width values for the clustering methods are as follows:

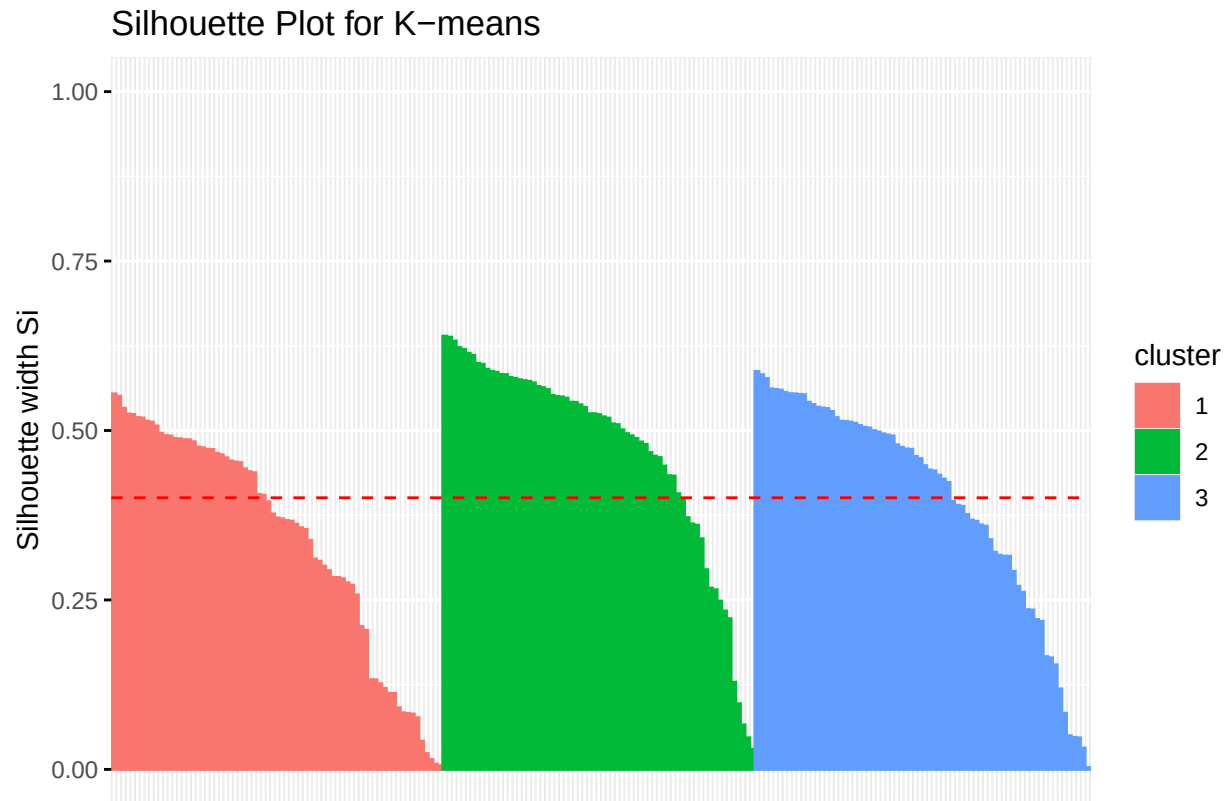
K-means: 0.40 K-means++: 0.40 K-medoids: 0.39

the clustering results suggest moderate separation among clusters, with K-means and K-means++ providing similar performance. K-medoids, while slightly less compact, remains robust to outliers, making it a good alternative for datasets with potential noise or outliers.

#SILHOUETTE PLOTS

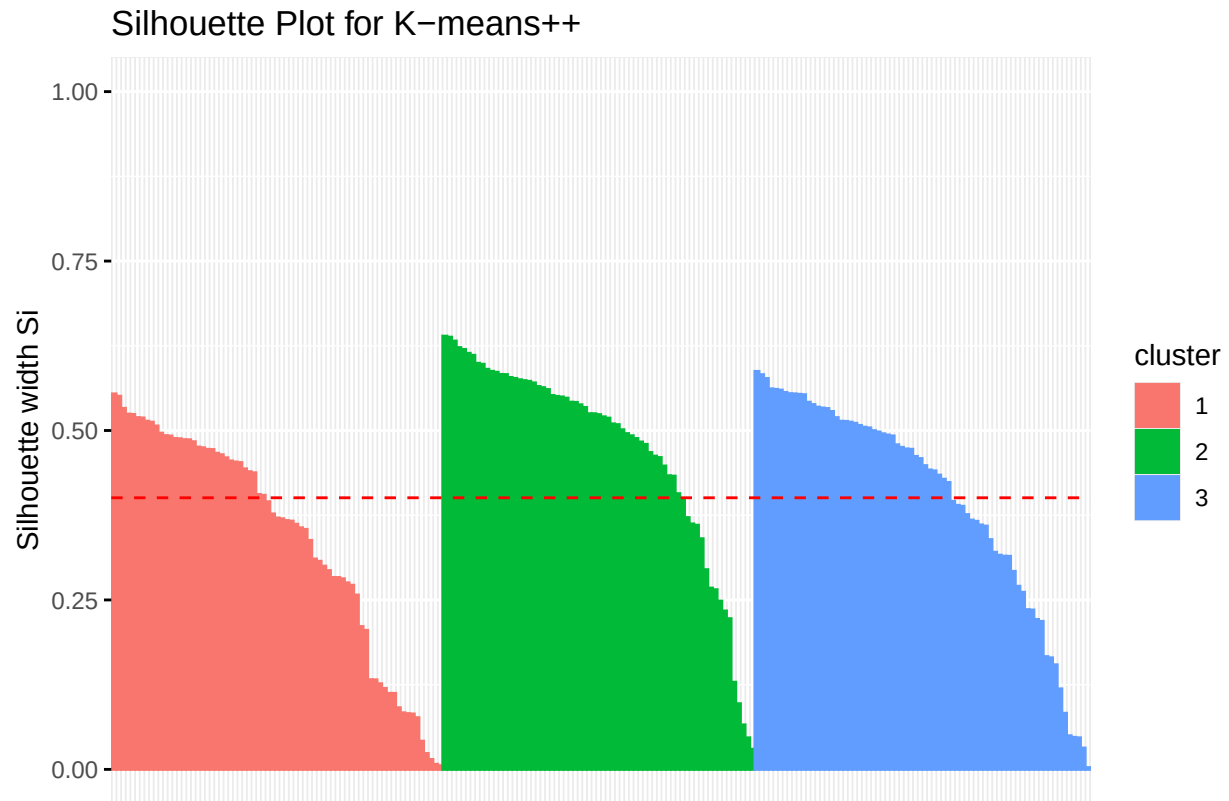
```
#silhouette plots
fviz_silhouette(sil_kmeans, main = "Silhouette Plot for K-means")
```

```
##   cluster size ave.sil.width
## 1      1    71          0.34
## 2      2    67          0.47
## 3      3    72          0.40
```

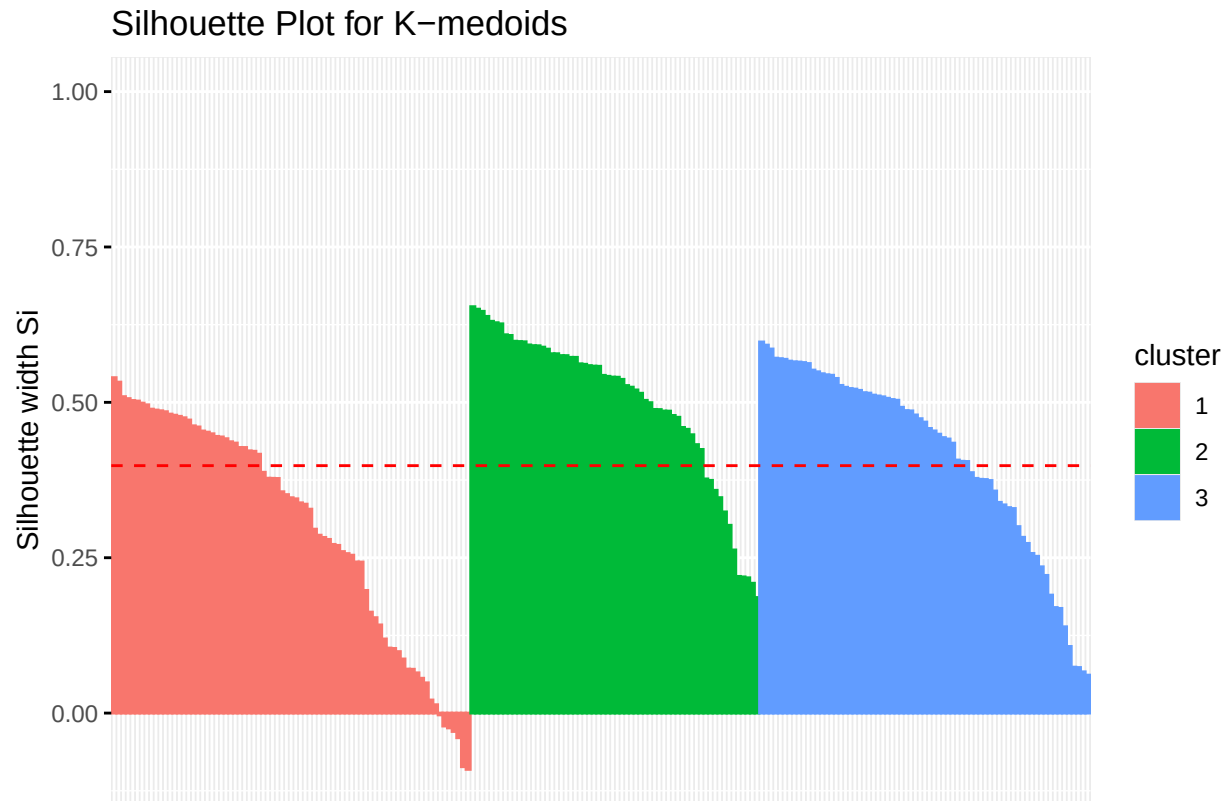
```
fviz_silhouette(sil_kmeans_plus, main = "Silhouette Plot for K-means++")
```

##	cluster	size	ave.sil.width
## 1	1	71	0.34
## 2	2	67	0.47
## 3	3	72	0.40



```
fviz_silhouette(sil_pam, main = "Silhouette Plot for K-medoids")
```

```
## cluster size ave.sil.width
## 1      1  77      0.30
## 2      2  62      0.50
## 3      3  71      0.42
```



K-means: The silhouette plot for K-means shows clusters with a moderate average silhouette width, indicating that while most points are well-clustered (positive silhouette values), some overlap and less-defined boundaries are present, particularly in clusters 1 and 3. The red dashed line marks the average silhouette score, highlighting the general clustering quality.

K-means++: The K-means++ silhouette plot closely resembles that of the standard K-means. The average silhouette width is similar, confirming that the improved initialization method helps achieve more reliable initial centroids but does not drastically change the cluster quality for this dataset. Slight improvements in cluster separation may be observed, enhancing stability during different runs.

K-medoids: The silhouette plot for K-medoids also shows a moderate clustering quality, similar to K-means and K-means++. However, the shape of the clusters and silhouette width distribution suggest that K-medoids might handle outliers better, as indicated by fewer points with very low silhouette scores. This method can be more robust, especially in datasets with potential noise or skewed distributions.

#CLUSTER STABILITY

```
# Analyze cluster stability and outlier management
outliers <- boxplot.stats(Seeds$`Asymmetry coefficient`)$out # Example for one variable
outliers
```

NULL

How else could we analyze cluster stability and outlier management??? Not sure how this works.

#Conclusion for the Research Question

In comparing K-means, K-means++, and K-medoids clustering on the SEEDS dataset, the following observations were made:

K-means and K-means++ produced similar results in terms of silhouette scores and cluster quality, with average silhouette widths around 0.40. This suggests that both methods performed moderately well in creating distinguishable clusters, with K-means++ offering more reliable initial centroids due to its improved initialization strategy. K-medoids, while showing a slightly lower average silhouette score (~ 0.39), demonstrated robustness against outliers, making it a good option for datasets prone to noise or with non-uniform distributions. The distribution of clusters and their characteristics indicated that all three methods were able to capture the underlying structure of the SEEDS dataset, which consists of three wheat varieties. Overall, K-means++ provided a balance of efficiency and stability, while K-medoids offered better handling of outliers.

For the SEEDS dataset, K-means++ appears to be the better method overall.